

STAT 8670 – Computational Methods in Statistics

2026 Fall Course Notes

Chi-Kuang Yeh

2026-09-12

Table of contents

Preface	6
Description	6
Prerequisites	6
Instructor	6
Office Hour	6
Grade Distribution	6
Assignment	7
Midterm	7
Project	7
Topics and Corresponding Lectures	7
Recommended Textbooks	7
Side Readings	8
1 Introduction to R	9
1.1 Why R?	9
1.2 The working environment	10
1.2.1 RStudio	11
1.2.2 R scripts and Quarto documents	12
1.2.3 A project-oriented workflow	13
1.3 R as a computing environment	14
1.4 Objects, data, and functions	15
1.5 Expressions, objects, and names	15
1.6 Data types	16
1.6.1 Changing data types	17
1.7 Operators	18
1.8 Vectorization	19
1.9 Recycling	20
1.10 Coercion	21
1.11 Missing and special values	22
1.12 Indexing and subsetting	23
1.13 Naming elements	25
1.14 Arrays and matrices	26
1.15 Lists and key-value pairs	28
1.16 Factors	30

1.17	Data frames	31
1.17.1	Inspecting a data frame	32
1.17.2	Subsetting a data frame	33
1.17.3	Adding and modifying variables	35
1.17.4	Importing rectangular data	35
1.18	Functions	36
1.18.1	Argument matching	37
1.18.2	Lazy evaluation	37
1.19	Environments and lexical scoping	38
1.20	Memory and copy-on-modify	39
1.21	Iteration: vectorization, loops, and the apply family	40
1.21.1	<code>apply()</code>	41
1.21.2	Why <code>apply()</code> can be dangerous for data frames	43
1.21.3	<code>lapply()</code>	44
1.21.4	<code>sapply()</code> and <code>vapply()</code>	45
1.21.5	<code>tapply()</code>	45
1.21.6	<code>Map()</code> and <code>mapply()</code>	45
1.21.7	Choosing an iteration tool	46
1.22	Packages and help	46
1.23	The tidyverse	47
1.23.1	Data frames and tibbles	47
1.23.2	Pipe operators	48
1.23.3	Core <code>dplyr</code> verbs	49
1.23.4	Applying functions across several columns	51
1.24	Base R and tidyverse are complementary	51
1.25	Numerical computation and finite precision	52
1.26	Randomness and reproducibility	53
1.27	Debugging and error messages	54
1.28	A computational workflow for this course	55
1.29	In-class exercises	56
1.30	Takeaways	58
1.31	Optional challenge	58
2	Computational Approaches	60
2.1	Theory versus Computation	61
2.2	Matrix Inversion	62
2.2.1	Multi-collinearity	64
2.2.2	Trade-off	66
3	Optimization	71
3.1	Why Numerical Optimization?	71
3.1.1	Visualizing the Objective Function	72
3.2	When There Is No Closed-Form Solution	74

3.3	Two Families of Optimization Methods	77
3.3.1	Deterministic Methods	77
3.3.2	Metaheuristic Methods	77
3.4	Local and Global Optima	78
3.5	Deterministic Numerical Methods	81
3.5.1	Root Finding	82
3.5.2	A Statistical Example: Normal Quantiles	82
3.5.3	Bisection Method	84
3.5.4	Visualizing Bisection	86
3.5.5	Newton-Raphson Method	87
3.5.6	Visualizing Newton's Method	90
3.5.7	Secant Method	92
3.6	Convergence	95
3.6.1	Comparing Convergence in R	95
3.7	From One Dimension to Multiple Dimensions	97
3.7.1	Visualizing a Two-Dimensional Objective	98
3.8	Gradient Descent	100
3.8.1	Visualizing the Optimization Path	102
3.8.2	Step Size Matters	103
3.9	Multivariate Newton's Method	105
3.10	Quasi-Newton Methods	106
3.11	Statistical Example: Logistic Regression	106
3.12	Scaling and Conditioning	109
3.13	Metaheuristic Methods	110
3.13.1	Why Accept a Worse Solution?	111
3.14	Simulated Annealing	111
3.14.1	Visualizing the Acceptance Probability	112
3.14.2	Simulated Annealing in R	114
3.14.3	Advantages of Simulated Annealing	116
3.14.4	Limitations of Simulated Annealing	116
3.15	Genetic Algorithm	116
3.15.1	Selection	117
3.15.2	Crossover	117
3.15.3	Mutation	117
3.15.4	Advantages	117
3.15.5	Limitations	117
3.16	Particle Swarm Optimization	118
3.16.1	Advantages	118
3.16.2	Limitations	118
3.17	Deterministic versus Metaheuristic Methods	119
3.18	Optimization Transfer	120
3.19	Computational Perspective	120
3.20	Questions to Think at Home	121

3.21 Takeaways	122
I Assignment	123
Assignment 1	124
Question 1: R Programming and Statistical Computing [10]	124
1a)	124
1b)	125
1c)	125
1d)	125
1e)	125
1f)	126
1g)	126
1h)	126
1i)	126
1j)	127
Question 2: Computational Results Using the Iris Data [10]	127
2a) [2]	127
2b) [2]	127
2c) [2]	127
2d) [2]	128
2e) [2]	128
Question 3: R Coding and Visualization [6]	128
3a) [3]	128
3b) [3]	129
Question 4: Numerical Linear Algebra [4]	129
4a) [2]	129
4b) [2]	130

Preface

Description

Topics included are optimization, numerical integration, bootstrapping, cross-validation and Jackknife, density estimation, smoothing, and use of the statistical computer package of S-plus/R.

Prerequisites

MATH 4752/6752 – Mathematical Statistics II, and the ability to program in a high-level language.

Instructor

[Chi-Kuang Yeh](#), I am an Assistant Professor in the Department of Mathematics and Statistics, [Georgia State University](#).

- Office: Suite 1407, 25 Park Place.
- Email: cych@gsu.edu.

Office Hour

9:00 - 12:00 on Wednesday or by appointment

Grade Distribution

- Assignments: 40%
- Exam: 30%
- Project: 30%
- Attendance: *7%

Assignment

- ☒ [Assignment 1](#): September 11, 2026
- ☐ Assignment 2: TBA

Midterm

- ☐ Exam 1: October 7, 2026
- ☐ Exam 2: TBA

Project

- ☐ Report: TBA
- ☐ Presentation: TBA

Topics and Corresponding Lectures

Those chapters are based on the lecture notes. This part will be updated frequently.

Status	Topic	Lecture Covered
	R Programming	1–3
	Numerical Approaches and Computational Statistics	4– 5
	Optimization	5–

Recommended Textbooks

- Givens, G.H. and Hoeting, J.A. (2012). *Computational Statistics*. Wiley, New York.
- Rizzo, M.L. (2007) *Statistical Computing with R*. CRC Press, Boca Raton.
- Hothorn, T. and Everitt, B.S. (2006). *A Handbook of Statistical Analyses Using R*. CRC Press, Boca Raton.

Side Readings

- Wickham, H., Çetinkaya-Rundel, M. and Grolemund, G. (2023). *R for Data Science*. O'Reilly.

1 Introduction to R

Learning objectives

By the end of this chapter, you should be able to:

1. Describe how R reads, evaluates, and prints an expression.
2. Distinguish an object from the name bound to that object.
3. Identify the major data types and data structures in R.
4. Predict how vectorization, coercion, recycling, and missing values affect a computation.
5. Create, inspect, index, and modify vectors, matrices, arrays, lists, and data frames.
6. Write and use simple R functions.
7. Choose appropriately among vectorization, loops, and the `apply()` family.
8. Use the tidyverse pipe and core `dplyr` verbs to express a data-analysis pipeline.
9. Write computations that are reproducible, readable, and numerically responsible.

1.1 Why R?

Modern statistical analyses routinely involve hundreds, thousands, or millions of observations. The calculations are too numerous, and often too complex, to perform reliably by hand. We therefore need software that can manage data, implement statistical methods, automate repeated computations, and document the complete analysis.

R is both a high-level programming language and an environment for statistical computing and graphics. It is especially useful for statistics because it is:

- **free and open source**, so students and researchers can use the same tools;
- **designed for data analysis**, with vectors, missing values, statistical models, and graphics built into the language;
- **extensible**, with a large ecosystem of packages developed by researchers and software developers;
- **reproducible**, because an analysis can be saved as code and rerun from beginning to end;

- **interoperable**, with interfaces to databases and languages such as C++, Python, and Julia.

R is not a menu-driven statistics program. Learning R means learning to express an analysis as a sequence of precise and reproducible computations.

i R, RStudio, and Quarto are different tools

R performs the computation. **RStudio** is an integrated development environment that helps us write and run code. **Quarto** combines text, code, results, figures, equations, and references into reproducible HTML, PDF, Word, or presentation output.

1.2 The working environment

There are different integrated development environment to use R, some of them are Rstudio, VS Code and Positron.

Rstudio

RStudio is an integrated development environment (IDE) designed specifically for working with the R programming language. It provides a user-friendly interface that includes a source editor, console, environment pane, and tools for plotting, debugging, version control, and package management. RStudio supports both R and Python and is widely used for data analysis, statistical modeling, and reproducible research. It also integrates seamlessly with tools like R Markdown, Shiny, and Quarto, making it popular among data scientists, statisticians, and educators.

Visual Studio Code (VS Code)

VS Code is a versatile code editor that supports multiple programming languages, including R. With the R extension for VS Code, users can write and execute R code, access R's console, and utilize features like syntax highlighting, code completion, and debugging. While not as specialized as RStudio for R development, VS Code offers a lightweight alternative with extensive customization options and support for various programming tasks.

Positron

Positron IDE is the next-generation integrated development environment developed by Posit, the company behind RStudio. Designed to be a modern, extensible, and language-agnostic IDE, Positron builds on the strengths of RStudio while supporting a broader range of languages and workflows, including R, Python, and Quarto.

1.2.1 RStudio

RStudio is an integrated development environment (IDE) for R, Python, and reproducible publishing. Its default layout contains four panes:

Pane	Main purpose
Source Console	Write and edit scripts and Quarto documents. Send expressions directly to R and view immediate results.
Environment/History	Inspect objects and review previous commands.
Files/Plots/Packages/Help/Viewer	Navigate files, inspect graphics, manage packages, read documentation, and preview output.

Use the console for short experiments. Any computation that matters should be placed in an R script or Quarto document so that it can be reproduced.

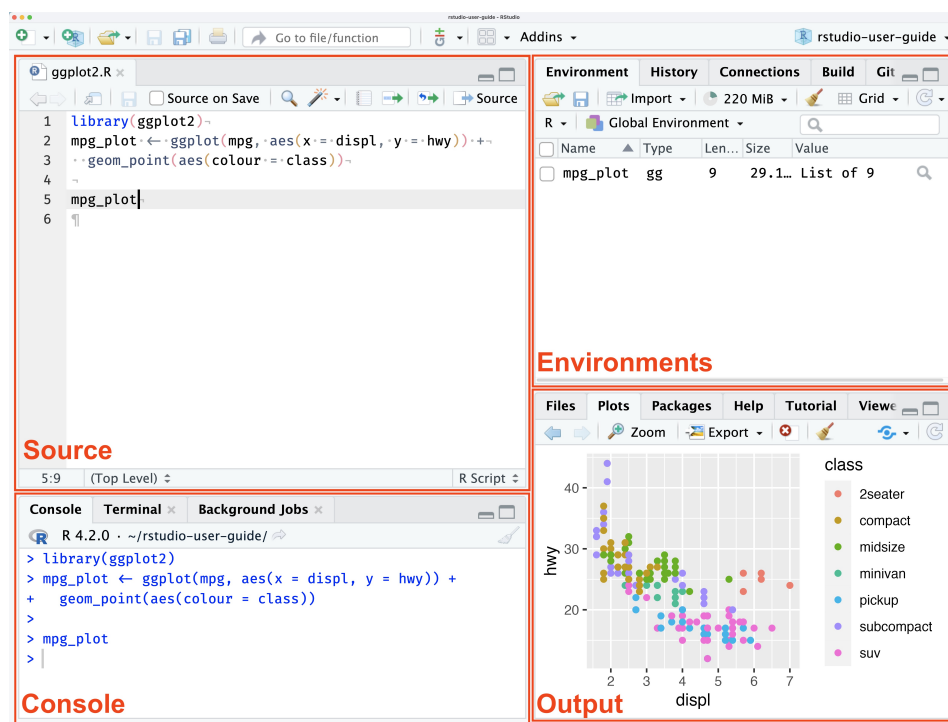


Figure 1.1: RStudio layout borrow from the Posit website.

1.2.2 R scripts and Quarto documents

An R script is a plain-text file with the extension `.R`. It contains R expressions that can be executed one line, one selection, or one file at a time.

A Quarto document has the extension `.qmd`. It combines:

- a YAML header describing the document;
- prose written in Markdown;
- executable code chunks;
- tables, equations, figures, citations, and cross-references.

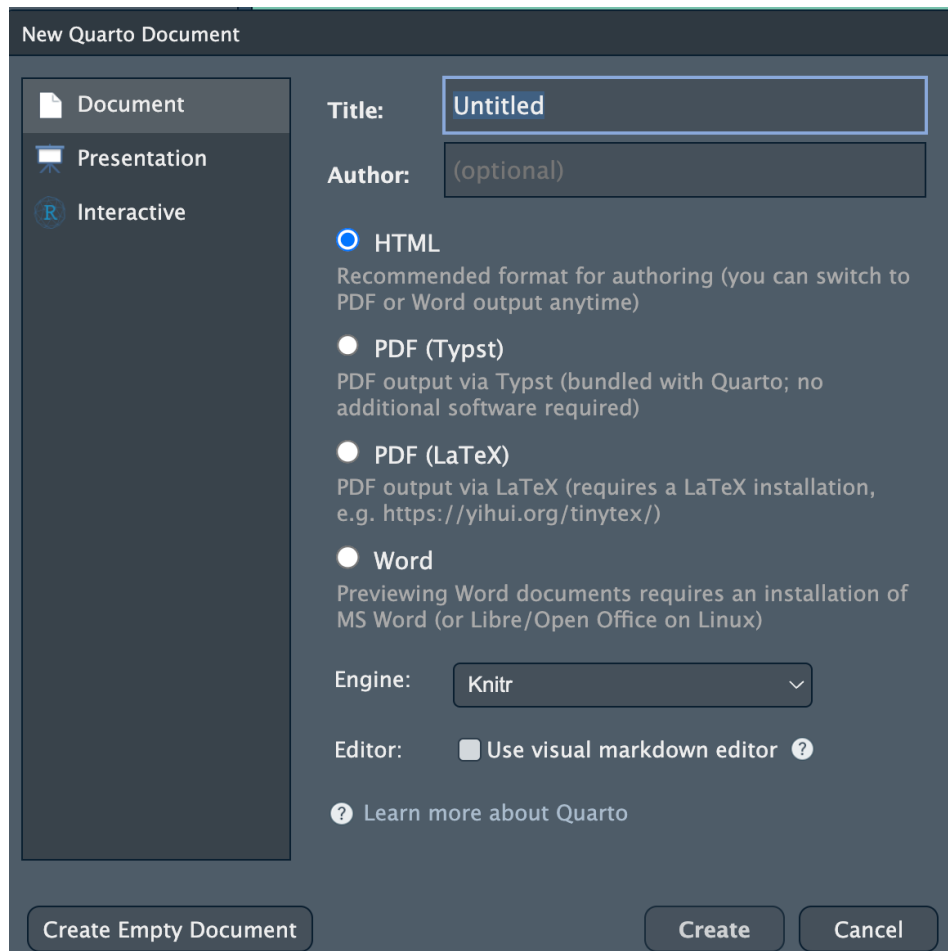


Figure 1.2: Quarto.

In Figure 1.3, the red box is the yaml, the yellow box is the executable code chunk and the green box is a figure.

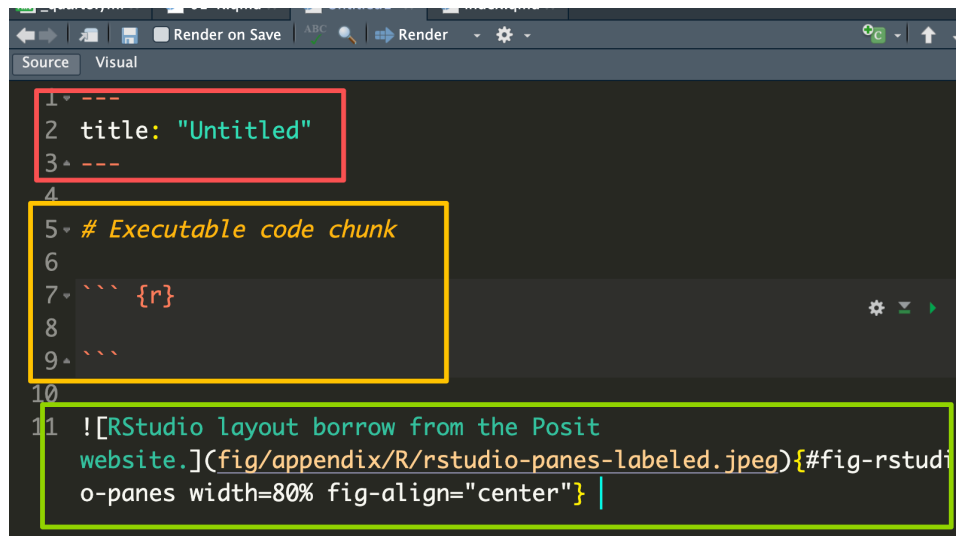


Figure 1.3: Quarto 2.

For example, an executable R chunk is written as

```
```{r}
x <- rnorm(100)
mean(x)
```
```

```
[1] 0.1211347
```

Rendering the document executes the enabled chunks in order and places their results into the final output.

1.2.3 A project-oriented workflow

Create one RStudio Project for the course or for each substantial analysis. Use project-relative paths such as `data/survey.csv` rather than hard-coded paths tied to one computer.

A simple project might contain:

```
stat8670-project/
  data/           # original and processed data
  R/              # reusable functions
  figures/        # saved figures
```

```
notes/          # Quarto lecture or analysis files
stat8670-project.Rproj
```

Avoid using `setwd()` inside a reproducible document. When a project is opened, its root provides a stable reference point for relative paths.

1.3 R as a computing environment

At the console, R follows a **read, evaluate, print loop** (REPL):

1. **Read:** parse the text entered by the user.
2. **Evaluate:** compute the value of the expression.
3. **Print:** display the result when appropriate.
4. **Loop:** wait for the next expression.

```
2 + 3 * 4
```

```
[1] 14
```

R respects operator precedence, so multiplication is performed before addition.

```
(2 + 3) * 4
```

```
[1] 20
```

Assignment usually returns its value invisibly:

```
x <- 2 + 3 * 4
x
```

```
[1] 14
```

A useful mental model

Most R code can be understood as **functions operating on objects**. Even operators such as `+` are functions.

```
`+`(2, 3)
```

```
[1] 5
```

1.4 Objects, data, and functions

Two fundamental kinds of objects that we will work with are:

1. **Data objects**
2. **Functions**

Data objects include numbers, vectors, matrices, arrays, lists, data frames, and fitted models.

A **function** is a set of instructions that receives input, performs a computation, and returns an output. Functions can be built into R, provided by a package, or written by the user.

```
mean(c(1, 2, 3, 4))
```

```
[1] 2.5
```

Here, `c(1, 2, 3, 4)` is a data object and `mean()` is a function.

1.5 Expressions, objects, and names

R computes with **objects**. A **name** is a symbol bound to an object.

```
sample_size <- 100
another_name <- sample_size

sample_size
```

```
[1] 100
```

```
another_name
```

```
[1] 100
```

The assignment operator `<-` creates or changes a binding.

Names are case-sensitive:

```
n <- 10
N <- 25

c(n = n, N = N)
```

```
  n  N
10 25
```

A useful naming convention is to separate words using `_`:

```
sample_mean <- 10
number_students <- 30
```

Use `ls()` to list names in the current environment and `rm()` to remove a binding.

```
temporary_result <- 42
"temporary_result" %in% ls()
```

```
[1] TRUE
```

```
rm(temporary_result)

"temporary_result" %in% ls()
```

```
[1] FALSE
```

1.6 Data types

The fundamental data structure in R is the **vector**. An atomic vector contains elements of one common type.

Common atomic types include:

| Type | Example | Typical use |
|-----------|----------------|---------------------------|
| logical | TRUE, FALSE | conditions and indicators |
| integer | 2L | counts and indices |
| double | 2, 3.14 | numerical computation |
| character | "R" | text and labels |
| complex | 1 + 2i | complex arithmetic |
| raw | charToRaw("R") | bytes |

```
logical_vector <- c(TRUE, FALSE, TRUE)
integer_vector <- c(1L, 2L, 3L)
double_vector <- c(1, 2, 3)
character_vector <- c("one", "two", "three")

typeof(logical_vector)
```

```
[1] "logical"
```

```
typeof(integer_vector)
```

```
[1] "integer"
```

```
typeof(double_vector)
```

```
[1] "double"
```

```
typeof(character_vector)
```

```
[1] "character"
```

1.6.1 Changing data types

Data types can be explicitly converted, but conversion may result in information loss.

```
x <- pi
x
```

```
[1] 3.141593
```

```
x_int <- as.integer(x)
x_int
```

```
[1] 3
```

Here, converting π to an integer removes its decimal component.

Common conversion functions include:

- `as.integer()`
- `as.numeric()`
- `as.character()`
- `as.logical()`
- `as.Date()`
- `as.factor()`
- `as.list()`
- `as.matrix()`
- `as.data.frame()`
- `as.vector()`
- `as.complex()`

 Type conversion can lose information

Always inspect the result after converting an object from one type to another.

1.7 Operators

A **unary operator** operates on one object:

```
-x
!x
```

A **binary operator** operates on two objects:

```
x + y
x - y
x * y
x / y
```

Comparison operators

Comparison operators return logical values:

```
x == y
x != y
x < y
x > y
x <= y
x >= y
```

Logical operators

Logical operators combine logical values:

```
x & y
x | y
!x
```

The operators `&` and `|` operate element by element.

```
x <- c(TRUE, FALSE, FALSE)
y <- c(TRUE, TRUE, FALSE)

x | y
```

```
[1] TRUE TRUE FALSE
```

```
x & y
```

```
[1] TRUE FALSE FALSE
```

By contrast, `&&` and `||` are intended for a single logical condition and are commonly used inside `if` statements.

1.8 Vectorization

Many R functions and operators act on entire vectors.

```
x <- c(1, 2, 3, 4)
```

```
x^2
```

```
[1] 1 4 9 16
```

```
sqrt(x)
```

```
[1] 1.000000 1.414214 1.732051 2.000000
```

```
sum(x)
```

```
[1] 10
```

```
mean(x)
```

```
[1] 2.5
```

Vectorized code often expresses statistical calculations more directly than element-by-element code.

1.9 Recycling

Arithmetic between vectors is usually element-wise.

```
x <- c(10, 20, 30)
```

```
y <- c(1, 2, 3)
```

```
x + y
```

```
[1] 11 22 33
```

If one vector is shorter, R may **recycle** its values.

```
c(10, 20, 30, 40) + c(1, 2)
```

```
[1] 11 22 31 42
```

When the shorter length does not divide the longer length exactly, R produces a warning.

```
c(10, 20, 30, 40) + c(1, 2, 3)
```

Warning in `c(10, 20, 30, 40) + c(1, 2, 3)`: longer object length is not a multiple of shorter object length

```
[1] 11 22 33 41
```

1.10 Coercion

An atomic vector must have one type. Combining different types may cause R to convert values to a common type.

```
mixed_numeric <- c(TRUE, 2L, 3.5)
mixed_character <- c(TRUE, 2L, 3.5, "four")

mixed_numeric
```

```
[1] 1.0 2.0 3.5
```

```
typeof(mixed_numeric)
```

```
[1] "double"
```

```
mixed_character
```

```
[1] "TRUE" "2"    "3.5"  "four"
```

```
typeof(mixed_character)
```

```
[1] "character"
```

A simplified coercion hierarchy is:

```
logical → integer → double → complex → character
```

Inspect unfamiliar objects using

```
typeof()  
class()  
length()  
str()
```

1.11 Missing and special values

R uses NA to represent a missing value.

```
day <- c("Monday", "Tuesday", "Wednesday", "Thursday", "Friday")  
weather <- c("Raining", "Sunny", NA, "Windy", "Snowing")  
  
weather_data <- data.frame(day, weather)  
weather_data
```

```
      day weather  
1  Monday Raining  
2  Tuesday  Sunny  
3 Wednesday  <NA>  
4  Thursday  Windy  
5   Friday Snowing
```

Missing values usually propagate through calculations:

```
scores <- c(88, NA, 91, 84)  
  
mean(scores)
```

```
[1] NA
```

```
mean(scores, na.rm = TRUE)
```

```
[1] 87.66667
```

```
is.na(scores)
```

```
[1] FALSE TRUE FALSE FALSE
```

Do not use

```
x == NA
```

to detect missing values. Use

```
is.na(x)
```

instead.

Other special numerical values include

```
c(1 / 0, -1 / 0, 0 / 0)
```

```
[1] Inf -Inf NaN
```

which produce `Inf`, `-Inf`, and `NaN`.

1.12 Indexing and subsetting

Indexing allows us to access or modify parts of an object.

! R indexing begins at 1

Unlike languages such as Python and C++, R uses **1-based indexing**. The first element is indexed by 1, not 0.

For a vector:

```
x <- c(first = 10, second = 20, third = 30, fourth = 40)
```

```
x[1]
```

```
first  
10
```

```
x[c(1, 3)]
```

```
first third  
  10    30
```

```
x[-2]
```

```
first  third fourth  
   10    30    40
```

```
x[x >= 30]
```

```
third fourth  
   30    40
```

```
x[c("second", "fourth")]
```

```
second fourth  
   20    40
```

Positive integers select positions, negative integers exclude positions, logical values filter positions, and character values select named elements.

 A common surprise

Prefer `seq_along(x)` to `1:length(x)` when iterating over the positions of an object.

```
empty <- numeric(0)
```

```
1:length(empty)
```

```
[1] 1 0
```

```
seq_along(empty)
```

```
integer(0)
```


1.13 Naming elements

Names can be attached to vector elements:

```
temp <- c(20, 30, 27, 31, 45)

names(temp) <- c("Mon", "Tues", "Wed", "Thurs", "Fri")

temp
```

| Mon | Tues | Wed | Thurs | Fri |
|-----|------|-----|-------|-----|
| 20 | 30 | 27 | 31 | 45 |

```
temp["Wed"]
```

```
Wed
27
```

A vector does not have rows, so this is inappropriate:

```
rownames(temp) <- "Day1"
```

A matrix can have row and column names:

```
temp_mat <- matrix(c(20, 30, 27, 31, 45),
                    nrow = 1,
                    ncol = 5)

colnames(temp_mat) <- c("Mon", "Tues", "Wed", "Thurs", "Fri")
rownames(temp_mat) <- "Day1"

temp_mat
```

| | Mon | Tues | Wed | Thurs | Fri |
|------|-----|------|-----|-------|-----|
| Day1 | 20 | 30 | 27 | 31 | 45 |

1.14 Arrays and matrices

An **array** is a multidimensional data structure.

```
array_1d <- array(1:10, dim = 10)
array_1d
```

```
[1]  1  2  3  4  5  6  7  8  9 10
```

```
array_2d <- array(1:12, dim = c(4, 3))
array_2d
```

```
      [,1] [,2] [,3]
[1,]     1     5     9
[2,]     2     6    10
[3,]     3     7    11
[4,]     4     8    12
```

```
array_3d <- array(1:24, dim = c(4, 3, 2))
array_3d
```

```
, , 1
```

```
      [,1] [,2] [,3]
[1,]     1     5     9
[2,]     2     6    10
[3,]     3     7    11
[4,]     4     8    12
```

```
, , 2
```

```
      [,1] [,2] [,3]
[1,]    13    17    21
[2,]    14    18    22
[3,]    15    19    23
[4,]    16    20    24
```

A **matrix** is a two-dimensional array:

```
my_matrix <- matrix(1:12, nrow = 4, ncol = 3)
my_matrix
```

```
      [,1] [,2] [,3]
[1,]     1     5     9
[2,]     2     6    10
[3,]     3     7    11
[4,]     4     8    12
```

```
is.matrix(array_2d)
```

```
[1] TRUE
```

```
is.matrix(my_matrix)
```

```
[1] TRUE
```

```
is.array(array_2d)
```

```
[1] TRUE
```

```
is.array(my_matrix)
```

```
[1] TRUE
```

A matrix is an atomic vector with a `dim` attribute, so all entries must share one common type.

```
A <- matrix(1:6, nrow = 2, ncol = 3)
```

```
attributes(A)
```

```
$dim
[1] 2 3
```

```
A[2, 3]
```

```
[1] 6
```

Matrix multiplication uses `%*%`:

```
B <- matrix(c(1, 0, 0, 1, 1, 1), nrow = 3)
```

```
A %*% B
```

```
      [,1] [,2]  
[1,]     1     9  
[2,]     2    12
```

while `*` performs element-wise multiplication.

1.15 Lists and key-value pairs

A list can contain objects of different types and lengths.

```
fit_summary <- list(  
  method = "least squares",  
  coefficients = c(intercept = 1.2, slope = 0.8),  
  converged = TRUE  
)  
  
str(fit_summary)
```

```
List of 3  
 $ method      : chr "least squares"  
 $ coefficients: Named num [1:2] 1.2 0.8  
 ..- attr(*, "names")= chr [1:2] "intercept" "slope"  
 $ converged    : logi TRUE
```

Named list elements naturally form **key-value pairs**.

```
key1 <- "Tues"
value1 <- 32

key2 <- "Wed"
value2 <- 28

list_temp <- list()

list_temp[[key1]] <- value1
list_temp[[key2]] <- value2

list_temp
```

```
$Tues
[1] 32
```

```
$Wed
[1] 28
```

Values can then be obtained from their keys:

```
list_temp[["Tues"]]
```

```
[1] 32
```

```
list_temp$Tues
```

```
[1] 32
```

For lists, note the important distinction:

```
fit_summary["coefficients"]
```

```
$coefficients
intercept      slope
      1.2        0.8
```

```
fit_summary[["coefficients"]]
```

```
intercept      slope  
      1.2      0.8
```

[returns a sublist, whereas [[extracts the element itself.

1.16 Factors

A factor represents categorical data using integer codes and a set of levels.

```
group <- factor(c("control", "treatment", "control"))
```

```
group
```

```
[1] control  treatment control  
Levels: control treatment
```

```
unclass(group)
```

```
[1] 1 2 1  
attr("levels")  
[1] "control" "treatment"
```

```
levels(group)
```

```
[1] "control" "treatment"
```

The order of factor levels matters in statistical models, so define it deliberately when necessary.

1.17 Data frames

A **data frame** is a two-dimensional rectangular data structure.

Conceptually, a data frame is a named list of equal-length vectors:

- each row represents an observational unit;
- each column represents a variable;
- columns may have different data types.

```
students <- data.frame(  
  id = 1:6,  
  name = c("Alice", "Bob", "Chen", "Divya", "Elena", "Farah"),  
  program = c("MS", "PhD", "MS", "PhD", "MS", "PhD"),  
  score = c(90, 85, NA, 94, 88, 91),  
  completed = c(TRUE, TRUE, FALSE, TRUE, TRUE, TRUE)  
)  
  
students
```

| | id | name | program | score | completed |
|---|----|-------|---------|-------|-----------|
| 1 | 1 | Alice | MS | 90 | TRUE |
| 2 | 2 | Bob | PhD | 85 | TRUE |
| 3 | 3 | Chen | MS | NA | FALSE |
| 4 | 4 | Divya | PhD | 94 | TRUE |
| 5 | 5 | Elena | MS | 88 | TRUE |
| 6 | 6 | Farah | PhD | 91 | TRUE |

Another familiar built-in data frame is:

```
iris <- datasets::iris  
head(iris)
```

| | Sepal.Length | Sepal.Width | Petal.Length | Petal.Width | Species |
|---|--------------|-------------|--------------|-------------|---------|
| 1 | 5.1 | 3.5 | 1.4 | 0.2 | setosa |
| 2 | 4.9 | 3.0 | 1.4 | 0.2 | setosa |
| 3 | 4.7 | 3.2 | 1.3 | 0.2 | setosa |
| 4 | 4.6 | 3.1 | 1.5 | 0.2 | setosa |
| 5 | 5.0 | 3.6 | 1.4 | 0.2 | setosa |
| 6 | 5.4 | 3.9 | 1.7 | 0.4 | setosa |

1.17.1 Inspecting a data frame

Always inspect newly imported data before analyzing them.

```
class(students)
```

```
[1] "data.frame"
```

```
dim(students)
```

```
[1] 6 5
```

```
nrow(students)
```

```
[1] 6
```

```
ncol(students)
```

```
[1] 5
```

```
names(students)
```

```
[1] "id"      "name"    "program" "score"   "completed"
```

```
str(students)
```

```
'data.frame':  6 obs. of  5 variables:
 $ id      : int  1 2 3 4 5 6
 $ name    : chr  "Alice" "Bob" "Chen" "Divya" ...
 $ program : chr  "MS" "PhD" "MS" "PhD" ...
 $ score   : num  90 85 NA 94 88 91
 $ completed: logi  TRUE TRUE FALSE TRUE TRUE TRUE
```

```
summary(students)
```


| | id | name | program | score | completed |
|----------|-------|-------------|-------------|--------------|---------------|
| Min. | :1.00 | Length :6 | Length :6 | Min. :85.0 | Mode :logical |
| 1st Qu.: | 2.25 | N.unique :6 | N.unique :2 | 1st Qu.:88.0 | FALSE:1 |
| Median | :3.50 | N.blank :0 | N.blank :0 | Median :90.0 | TRUE :5 |
| Mean | :3.50 | Min.nchar:3 | Min.nchar:2 | Mean :89.6 | |
| 3rd Qu.: | 4.75 | Max.nchar:5 | Max.nchar:3 | 3rd Qu.:91.0 | |
| Max. | :6.00 | | | Max. :94.0 | |
| | | | | NAs :1 | |

```
head(students, 3)
```

```

  id name program score completed
1  1 Alice      MS    90        TRUE
2  2  Bob      PhD    85        TRUE
3  3  Chen      MS    NA        FALSE

```

`str()` is especially useful because it gives a compact summary of the object's structure and types.

1.17.2 Subsetting a data frame

Use

```
data[rows, columns]
```

for two-dimensional indexing:

```
students[1, ]
```

```

  id name program score completed
1  1 Alice      MS    90        TRUE

```

```
students[, c("name", "score")]
```

```

  name score
1 Alice    90
2  Bob     85
3  Chen    NA
4 Divya    94
5 Elena    88
6 Farah    91

```

```
students[students$program == "PhD", ]
```

| | id | name | program | score | completed |
|---|----|-------|---------|-------|-----------|
| 2 | 2 | Bob | PhD | 85 | TRUE |
| 4 | 4 | Divya | PhD | 94 | TRUE |
| 6 | 6 | Farah | PhD | 91 | TRUE |

```
students[1:3, c("name", "score")]
```

| | name | score |
|---|-------|-------|
| 1 | Alice | 90 |
| 2 | Bob | 85 |
| 3 | Chen | NA |

```
students[2, "score"]
```

```
[1] 85
```

Three commonly used ways to extract a column are:

```
students["score"]
```

| | score |
|---|-------|
| 1 | 90 |
| 2 | 85 |
| 3 | NA |
| 4 | 94 |
| 5 | 88 |
| 6 | 91 |

```
students[["score"]]
```

```
[1] 90 85 NA 94 88 91
```

```
students$score
```

```
[1] 90 85 NA 94 88 91
```

The first returns a one-column data frame. The latter two return the underlying vector.

1.17.3 Adding and modifying variables

```
students$passed <- students$score >= 70

students$centered_score <-
  students$score - mean(students$score, na.rm = TRUE)

students
```

| | id | name | program | score | completed | passed | centered_score |
|---|----|-------|---------|-------|-----------|--------|----------------|
| 1 | 1 | Alice | MS | 90 | TRUE | TRUE | 0.4 |
| 2 | 2 | Bob | PhD | 85 | TRUE | TRUE | -4.6 |
| 3 | 3 | Chen | MS | NA | FALSE | NA | NA |
| 4 | 4 | Divya | PhD | 94 | TRUE | TRUE | 4.4 |
| 5 | 5 | Elena | MS | 88 | TRUE | TRUE | -1.6 |
| 6 | 6 | Farah | PhD | 91 | TRUE | TRUE | 1.4 |

Missingness propagates naturally. If a score is unknown, whether the student passed is also unknown.

1.17.4 Importing rectangular data

```
survey <- read.csv(
  "data/survey.csv",
  na.strings = c("", "NA", ".")
)

measurements <- read.table(
  "data/measurements.txt",
  header = TRUE,
  sep = "\t"
)
```

After importing, check:

```
dim(survey)
names(survey)
str(survey)
head(survey)
colSums(is.na(survey))
```

Data import is part of the analysis

Import settings such as delimiters, missing-value codes, decimal marks, encodings, and inferred variable types can change the data that R sees and therefore affect the final analysis.

Do not automatically treat a data frame as a matrix

`apply()` first converts a data frame to a matrix. A mixed-type data frame can therefore be converted entirely to character values. Use `lapply()`, `vapply()`, or `dplyr::across()` for column-wise operations when types differ.

1.18 Functions

A function packages a computation so that it can be reused.

```
standardize <- function(x, remove_na = FALSE) {  
  center <- mean(x, na.rm = remove_na)  
  spread <- sd(x, na.rm = remove_na)  
  
  (x - center) / spread  
}  
  
standardize(c(2, 4, 6, 8))
```

```
[1] -1.1618950 -0.3872983  0.3872983  1.1618950
```

A function has three main components:

- **formals:** the arguments;
- **body:** the expressions evaluated;
- **environment:** where the function was created.

```
formals(standardize)
```

```
$x
```

```
$remove_na  
[1] FALSE
```

```
body(standardize)
```

```
{  
  center <- mean(x, na.rm = remove_na)  
  spread <- sd(x, na.rm = remove_na)  
  (x - center)/spread  
}
```

```
environment(standardize)
```

```
<environment: R_GlobalEnv>
```

When a function is called, R creates a temporary environment, matches the arguments, evaluates the body, and returns the value of the final expression unless `return()` is called earlier.

1.18.1 Argument matching

Arguments can be matched by name or position.

```
round(x = 3.14159, digits = 3)
```

```
[1] 3.142
```

```
round(3.14159, 3)
```

```
[1] 3.142
```

Named arguments are often clearer in code intended to be read by others.

1.18.2 Lazy evaluation

R evaluates function arguments only when they are needed.

```
choose_one <- function(x, y) {  
  x  
}  
  
choose_one(  
  10,  
  stop("This argument is never evaluated")  
)
```

```
[1] 10
```

This behavior is called **lazy evaluation**.

1.19 Environments and lexical scoping

An environment stores name-object bindings and has a parent environment.

R uses **lexical scoping**: functions look for nonlocal names in the environment where they were defined.

```
make_power <- function(power) {  
  function(x) {  
    x^power  
  }  
}  
  
square <- make_power(2)  
cube <- make_power(3)  
  
square(5)
```

```
[1] 25
```

```
cube(5)
```

```
[1] 125
```

A function together with its defining environment is called a **closure**.

Local assignment affects the current function environment:

```
x <- 100

change_locally <- function() {
  x <- 5
  x
}

change_locally()
```

```
[1] 5
```

```
x
```

```
[1] 100
```

1.20 Memory and copy-on-modify

R behaves as if objects are copied when modified.

```
x <- c(10, 20, 30)
y <- x

y[1] <- 999

x
```

```
[1] 10 20 30
```

```
y
```

```
[1] 999 20 30
```

Changing `y` does not change `x`.

Repeatedly expanding an object can require many allocations:

```
result <- numeric(0)

for (i in seq_len(100000)) {
  result <- c(result, i^2)
}
```

Instead, preallocate the required memory:

```
n <- 100000

result <- numeric(n)

for (i in seq_len(n)) {
  result[i] <- i^2
}

head(result)
```

```
[1] 1 4 9 16 25 36
```

1.21 Iteration: vectorization, loops, and the apply family

An iterative computation can usually be expressed in several ways.

```
x <- c(2, 4, 6, 8)

# Vectorized
x^2
```

```
[1] 4 16 36 64
```

```
# Preallocated loop
squares <- numeric(length(x))

for (i in seq_along(x)) {
  squares[i] <- x[i]^2
}

squares
```



```
[1]  4 16 36 64
```

```
# Apply a function
vapply(
  x,
  function(value) value^2,
  numeric(1)
)
```

```
[1]  4 16 36 64
```

Use vectorization when an existing vectorized function directly expresses the computation. Use a loop when the calculation has evolving state or complex indexing. Use an apply-family function when the same operation is independently applied to several pieces of an object.

! The apply family is not magic vectorization

`apply()` and `lapply()` still perform iteration internally. They are not automatically faster than a well-written preallocated `for` loop. Their main advantage is often clarity and conciseness.

1.21.1 `apply()`

`apply(X, MARGIN, FUN, ...)` applies a function over dimensions of a matrix or array.

- `MARGIN = 1`: rows
- `MARGIN = 2`: columns

```
X <- matrix(1:12, nrow = 3, byrow = TRUE)

colnames(X) <- c("x1", "x2", "x3", "x4")

X
```

```
      x1 x2 x3 x4
[1,]  1  2  3  4
[2,]  5  6  7  8
[3,]  9 10 11 12
```

```
apply(X, MARGIN = 1, FUN = mean)
```

```
[1] 2.5 6.5 10.5
```

```
apply(X, MARGIN = 2, FUN = sd)
```

```
x1 x2 x3 x4  
4 4 4 4
```

Additional arguments are passed through ...:

```
X_missing <- X  
X_missing[2, 3] <- NA  
  
apply(X_missing, 2, mean, na.rm = TRUE)
```

```
x1 x2 x3 x4  
5 6 7 8
```

For common operations, specialized functions are preferable:

```
rowMeans(X)
```

```
[1] 2.5 6.5 10.5
```

```
colMeans(X)
```

```
x1 x2 x3 x4  
5 6 7 8
```

```
rowSums(X)
```

```
[1] 10 26 42
```

```
colSums(X)
```

```
x1 x2 x3 x4  
15 18 21 24
```

1.21.2 Why apply() can be dangerous for data frames

```
mixed_df <- data.frame(  
  id = c("A", "B", "C"),  
  x = c(10, 20, 30),  
  y = c(2, 4, 6)  
)  
  
as.matrix(mixed_df)
```

```
      id  x  y  
[1,] "A" 10 2  
[2,] "B" 20 4  
[3,] "C" 30 6
```

```
typeof(as.matrix(mixed_df))
```

```
[1] "character"
```

Because one column is character, the entire matrix becomes character.

A safer approach is:

```
numeric_columns <- vapply(  
  mixed_df,  
  is.numeric,  
  logical(1)  
)  
  
vapply(  
  mixed_df[numeric_columns],  
  mean,  
  numeric(1)  
)
```

```
  x  y  
20 4
```

1.21.3 lapply()

`lapply()` applies a function to each element and always returns a list.

```
lapply(students, class)
```

```
$id
[1] "integer"

$name
[1] "character"

$program
[1] "character"

$score
[1] "numeric"

$completed
[1] "logical"

$passed
[1] "logical"

$centered_score
[1] "numeric"
```

```
lapply(
  students[c("score", "centered_score")],
  mean,
  na.rm = TRUE
)
```

```
$score
[1] 89.6

$centered_score
[1] 5.861978e-15
```

1.21.4 `sapply()` and `vapply()`

`sapply()` attempts to simplify the output.

```
sapply(students, class)
```

| | | | | |
|-----------|----------------|-------------|-----------|-----------|
| id | name | program | score | completed |
| "integer" | "character" | "character" | "numeric" | "logical" |
| passed | centered_score | | | |
| "logical" | "numeric" | | | |

`vapply()` requires the expected output type:

```
vapply(students, typeof, character(1))
```

| | | | | |
|-----------|----------------|-------------|----------|-----------|
| id | name | program | score | completed |
| "integer" | "character" | "character" | "double" | "logical" |
| passed | centered_score | | | |
| "logical" | "double" | | | |

This makes `vapply()` safer for reusable code.

1.21.5 `tapply()`

`tapply()` splits a vector into groups and applies a function within each group.

```
with(
  students,
  tapply(score, program, mean, na.rm = TRUE)
)
```

```
MS PhD
89  90
```

1.21.6 `Map()` and `mapply()`

These functions apply a function to corresponding elements of multiple inputs.

```
Map(
  function(center, spread) rnorm(3, center, spread),
  center = c(0, 10),
  spread = c(1, 2)
)
```

```
[[1]]
[1] 0.9444969 -0.3233628 -1.3212824

[[2]]
[1] 7.857089 11.570026 8.406937
```

1.21.7 Choosing an iteration tool

| Goal | Recommended starting point |
|---|--|
| Element-wise arithmetic | Vectorized function or operator |
| Standard matrix row/column summaries | <code>rowSums()</code> , <code>colSums()</code> , <code>rowMeans()</code> ,
<code>colMeans()</code> |
| General matrix row/column operation | <code>apply()</code> |
| Operation on list or data-frame columns | <code>lapply()</code> |
| Known scalar output type | <code>vapply()</code> |
| Grouped operation on one vector | <code>tapply()</code> |
| Data-frame columns in tidyverse | <code>dplyr::across()</code> |
| Complex state-dependent computation | Preallocated for loop |

1.22 Packages and help

Packages extend R with functions, data, documentation, and statistical methods. The historical package count can be found <https://www.datasciencemeta.com/rpackages>.

```
# CRAN
install.packages("packageName")

# Bioconductor
BiocManager::install("packageName")

# GitHub
remotes::install_github("username/repository")
```

Load an installed package:

```
library(packageName)
```

or call one function explicitly:

```
packageName::function_name()
```

Useful help tools include:

```
?mean  
help("mean")  
example(mean)  
args(mean)  
methods(mean)
```

1.23 The tidyverse

The **tidyverse** is a coordinated collection of R packages for data manipulation, visualization, importing, tidying, and programming.

```
install.packages("tidyverse")
```

```
library(tidyverse)
```

Important packages include:

```
library(dplyr)    # data manipulation  
library(ggplot2)  # data visualization  
library(readr)    # data import  
library(tibble)   # tidy data frames  
library(tidyr)    # data tidying  
library(purrr)    # functional programming
```

Base R and tidyverse tools are complementary rather than competing systems.

1.23.1 Data frames and tibbles

A **tibble** is a modern data frame.

```
students_tbl <- as_tibble(students)
```

```
students_tbl
```

```
# A tibble: 6 x 7
```

```
      id name  program score completed passed centered_score
  <int> <chr> <chr>   <dbl> <lgl>    <lgl>         <dbl>
1     1  Alice  MS       90  TRUE    TRUE         0.400
2     2   Bob  PhD       85  TRUE    TRUE        -4.60
3     3  Chen  MS       NA FALSE    NA           NA
4     4 Divya  PhD       94  TRUE    TRUE         4.40
5     5  Elena  MS       88  TRUE    TRUE        -1.60
6     6  Farah  PhD       91  TRUE    TRUE         1.40
```

```
class(students_tbl)
```

```
[1] "tbl_df"      "tbl"        "data.frame"
```

Some important differences are:

| Operation | Base data frame | Tibble |
|---------------------------|----------------------------|------------------|
| Printing | May print many rows | Compact preview |
| <code>x[, "score"]</code> | May simplify | Remains a tibble |
| Partial matching | May occur in some contexts | More strict |
| Row names | Supported | Discouraged |

Both are still data frames:

```
is.data.frame(students_tbl)
```

```
[1] TRUE
```

1.23.2 Pipe operators

The native R pipe is


```
|>
```

and the commonly used magrittr/tidyverse pipe is

```
%>%
```

The pipe passes the result of one operation to the next function.

```
set.seed(777)

x <- rnorm(5)

# Without pipe
print(round(mean(x), 2))
```

```
[1] 0.37
```

```
# Native pipe
x |>
  mean() |>
  round(2) |>
  print()
```

```
[1] 0.37
```

The pipe makes the sequence of operations explicit.

A useful formatting rule is that `|>` should have spaces around it and normally appear at the end of a line.

1.23.3 Core dplyr verbs

| Verb | Purpose |
|--------------------------|-----------------------------------|
| <code>select()</code> | Choose or reorder columns |
| <code>filter()</code> | Retain rows satisfying conditions |
| <code>arrange()</code> | Sort observations |
| <code>mutate()</code> | Create or modify variables |
| <code>summarise()</code> | Compute summaries |

| Verb | Purpose |
|-------------------------|---------------|
| <code>group_by()</code> | Define groups |

Example:

```
students_tbl |>
  filter(completed, !is.na(score)) |>
  select(name, program, score) |>
  arrange(desc(score))
```

```
# A tibble: 5 x 3
  name  program score
<chr> <chr>   <dbl>
1 Divya PhD      94
2 Farah PhD      91
3 Alice MS       90
4 Elena MS       88
5 Bob   PhD       85
```

Create variables and summarize by group:

```
students_tbl |>
  mutate(
    passed = score >= 70,
    score_z =
      (score - mean(score, na.rm = TRUE)) /
      sd(score, na.rm = TRUE)
  ) |>
  group_by(program) |>
  summarise(
    n = n(),
    n_observed = sum(!is.na(score)),
    mean_score = mean(score, na.rm = TRUE),
    sd_score = sd(score, na.rm = TRUE),
    .groups = "drop"
  )
```

```
# A tibble: 2 x 5
  program      n n_observed mean_score sd_score
<chr>   <int>     <int>     <dbl>   <dbl>
1 MS         3         2         89     1.41
2 PhD        3         3         90     4.58
```

1.23.4 Applying functions across several columns

Inside `mutate()` or `summarise()`, `across()` applies functions to selected columns.

```
students_tbl |>
  summarise(
    across(
      where(is.numeric),
      list(
        mean = ~ mean(.x, na.rm = TRUE),
        sd = ~ sd(.x, na.rm = TRUE)
      ),
      .names = "{.col}_{.fn}"
    )
  )
```

A tibble: 1 x 6

| | id_mean | id_sd | score_mean | score_sd | centered_score_mean | centered_score_sd |
|---|---------|-------|------------|----------|---------------------|-------------------|
| | <dbl> | <dbl> | <dbl> | <dbl> | <dbl> | <dbl> |
| 1 | 3.5 | 1.87 | 89.6 | 3.36 | 5.86e-15 | 3.36 |

This provides a tidyverse alternative to column-wise `lapply()` or `vapply()`.

1.24 Base R and tidyverse are complementary

For example, the same grouped mean can be obtained using either style.

```
# Base R
aggregate(
  score ~ program,
  data = students,
  FUN = mean,
  na.rm = TRUE
)
```

| | program | score |
|---|---------|-------|
| 1 | MS | 89 |
| 2 | PhD | 90 |

```
# tidyverse
students_tbl |>
  group_by(program) |>
  summarise(
    mean_score = mean(score, na.rm = TRUE),
    .groups = "drop"
  )
```

```
# A tibble: 2 x 2
  program mean_score
  <chr>      <dbl>
1 MS          89
2 PhD          90
```

The important questions are the same regardless of syntax:

- What object enters the function?
- What object does the function return?
- How are missing values handled?
- Are types and groups preserved?

1.25 Numerical computation and finite precision

Most real numbers cannot be represented exactly using binary floating-point arithmetic.

```
0.1 + 0.2
```

```
[1] 0.3
```

```
(0.1 + 0.2) == 0.3
```

```
[1] FALSE
```

```
all.equal(0.1 + 0.2, 0.3)
```

```
[1] TRUE
```

For computed floating-point quantities, tolerance-based comparisons such as `all.equal()` are usually more appropriate than exact equality.

Numerical algorithms must also consider overflow, underflow, cancellation, conditioning, and convergence.

```
log(dbinom(500, size = 1000, prob = 0.01))
```

```
[1] -Inf
```

```
dbinom(  
  500,  
  size = 1000,  
  prob = 0.01,  
  log = TRUE  
)
```

```
[1] -1618.143
```

The second computation is numerically safer.

1.26 Randomness and reproducibility

Pseudo-random numbers are generated deterministically from an internal state.

```
set.seed(8670)  
rnorm(5)
```

```
[1] -0.4916281  1.1467524  1.5963118 -0.6655463  0.5516107
```

```
set.seed(8670)  
rnorm(5)
```

```
[1] -0.4916281  1.1467524  1.5963118 -0.6655463  0.5516107
```

The two results are identical.

Set the seed at a meaningful boundary, usually once near the beginning of a simulation, rather than repeatedly inside the simulation loop.

Record the information necessary to reproduce the analysis:

```
sessionInfo()
```

```
R version 4.6.0 (2026-04-24)
Platform: aarch64-apple-darwin23
Running under: macOS Tahoe 26.6.2
```

```
Matrix products: default
```

```
BLAS:   /Library/Frameworks/R.framework/Versions/4.6/Resources/lib/libRblas.0.dylib
LAPACK: /Library/Frameworks/R.framework/Versions/4.6/Resources/lib/libRlapack.dylib; LAPACK
```

```
locale:
```

```
[1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
```

```
time zone: America/New_York
```

```
tzcode source: internal
```

```
attached base packages:
```

```
[1] stats      graphics  grDevices  utils      datasets  methods    base
```

```
other attached packages:
```

```
[1] lubridate_1.9.5 forcats_1.0.1  stringr_1.6.0  dplyr_1.2.1
[5] purrr_1.2.2     readr_2.2.0    tidyr_1.3.2    tibble_3.3.1
[9] ggplot2_4.0.3   tidyverse_2.0.0
```

```
loaded via a namespace (and not attached):
```

```
[1] gtable_0.3.6      jsonlite_2.0.0    compiler_4.6.0    tidyselect_1.2.1
[5] scales_1.4.0      yaml_2.3.12       fastmap_1.2.0     R6_2.6.1
[9] generics_0.1.4    knitr_1.51        pillar_1.11.1     RColorBrewer_1.1-
3
[13] tzdb_0.5.0        rlang_1.3.0       utf8_1.2.6        stringi_1.8.9
[17] xfun_0.60         S7_0.2.2          otel_0.2.0        timechange_0.4.0
[21] cli_3.6.6         withr_3.0.3       magrittr_2.0.5    digest_0.6.39
[25] grid_4.6.0        rstudioapi_0.19.0 hms_1.1.4         lifecycle_1.0.5
[29] vctrs_0.7.3       evaluate_1.0.5    glue_1.8.1        farver_2.1.2
[33] codetools_0.2-20  rmarkdown_2.31    tools_4.6.0       pkgconfig_2.0.3
[37] htmltools_0.5.9
```

1.27 Debugging and error messages

When an error occurs:

1. Read the entire error message.
2. Reduce the problem to the smallest example that still fails.
3. Inspect objects with `str()`, `typeof()`, `class()`, and `dim()`.
4. Use `traceback()` after an error when necessary.
5. Verify assumptions with functions such as `stopifnot()`.

```
safe_mean <- function(x) {  
  stopifnot(  
    is.numeric(x),  
    length(x) > 0  
  )  
  
  mean(x)  
}  
  
safe_mean(c(2, 4, 6))
```

```
[1] 4
```

1.28 A computational workflow for this course

A reliable statistical computation separates:

1. **Inputs:** data, constants, configuration, and random seeds.
2. **Transformation:** cleaning and constructing analysis variables.
3. **Computation:** algorithms, models, optimization, or simulation.
4. **Validation:** diagnostics, tests, and sensitivity analysis.
5. **Communication:** figures, tables, interpretation, and limitations.

For example:

```
set.seed(8670)  
  
n <- 200  
  
x <- rnorm(n)  
y <- 1 + 2 * x + rnorm(n, sd = 0.75)  
  
simulated_data <- data.frame(  
  x = x,  
  y = y
```

```
)

fit <- lm(y ~ x, data = simulated_data)

coef(fit)
```

```
(Intercept)          x
    1.020794    1.991843
```

This script records where the data came from, fixes the random seed, creates an explicit data object, and passes that object to the statistical model.

1.29 In-class exercises

Predict the value and type of each expression before running the code.

```
c(TRUE, 2L, 3.5)

c(1, 2, 3, 4) + c(10, 20)

c(1, 2, NA) > 1

TRUE | NA
```

Using

```
x <- c(4, NA, 10, 15, 3, NA, 21)
```

write expressions that return:

1. all nonmissing values;
2. values greater than or equal to 10;
3. the positions of values greater than or equal to 10;
4. the mean of the observed values.

Without running the code, determine the result of `f(5)` and identify where each occurrence of `a` is found.


```

a <- 10

f <- function(x) {
  a <- 2

  g <- function(y) {
    a * y
  }

  g(x)
}

f(5)

```

Write a function that takes a positive integer n and returns a vector whose i th element is

$$\frac{(-1)^{i+1}}{i}.$$

Preallocate the result and use `seq_len(n)`.

Using `students`, complete each task once with base R and once with `dplyr`:

1. retain students with observed scores;
2. select `name`, `program`, and `score`;
3. create a variable equal to `score - 90`;
4. sort from highest to lowest score;
5. compute the number of observed scores and mean score within each program.

Before running the code, predict whether each result is a vector, data frame, tibble, or grouped tibble.

For each task, choose among a vectorized function, `apply()`, `lapply()`, `vapply()`, `tapply()`, `dplyr::across()`, or a loop. Justify your choice and then write the code.

1. Compute the mean of every numeric column in `students`.
2. Compute the row sums of a $10,000 \times 50$ numerical matrix.
3. Record the class of every column in a data frame.
4. Compute the mean score within each program.
5. Fit the same model to each of 20 independently generated data sets and retain the fitted-model objects.

Predict the type and contents of `result`. Explain why `apply()` is inappropriate and rewrite the computation using `vapply()` or `dplyr::across()`.

```
dat <- data.frame(
  subject = c("A", "B", "C"),
  pre = c(10, 12, 9),
  post = c(14, 13, 15)
)

result <- apply(dat, 2, mean)
```

1.30 Takeaways

- R is both a programming language and an environment for statistical computing.
- Objects and functions are the basic building blocks of R programs.
- Atomic vectors are the foundation of R computation.
- R uses **1-based indexing**.
- Vectorization, recycling, coercion, and missing-value propagation are central features of R.
- Matrices and arrays require homogeneous element types, whereas lists can contain heterogeneous objects.
- A data frame is a named list of equal-length columns.
- Tibbles are modern data frames with stricter and more predictable behavior.
- Functions have arguments, a body, and an environment.
- R uses lexical scoping and lazy evaluation.
- The apply family provides convenient abstractions for repeated computation but is not automatically faster than loops.
- `apply()` is designed primarily for matrices and arrays and can cause unwanted coercion when applied to mixed-type data frames.
- Base R and tidyverse tools are complementary.
- The pipe (`%>%` or `|>`) can make a sequence of data transformations easier to read.
- R's copy-on-modify behavior gives objects intuitive semantics, although memory allocation still matters.
- Statistical computing requires attention to floating-point arithmetic, missing values, reproducibility, and numerical stability.
- Understanding R's computational model makes code easier to predict, debug, optimize, and trust.

1.31 Optional challenge

Explain why these two functions may have very different memory and timing behavior even though they return the same values.

```

grow <- function(n) {
  out <- numeric(0)

  for (i in seq_len(n)) {
    out <- c(out, i^2)
  }

  out
}

preallocate <- function(n) {
  out <- numeric(n)

  for (i in seq_len(n)) {
    out[i] <- i^2
  }

  out
}

```

Design a timing experiment for increasing values of `n`.

Some of the materials are adapted from [CMU Stat36-350](#).

A comprehensive reference for the *tidyverse* tools is [R for Data Science](#).

A comprehensive reference for *ggplot2* is [ggplot2: Elegant Graphics for Data Analysis](#).

Additional introductory material is adapted from [Why R?](#).

2 Computational Approaches

“Statistical computing is more than programming. It is the process of using computation to connect statistical theory with practical data analysis. Programming provides the tools, but the main goal is to understand how a statistical method can be implemented, approximated, validated, and interpreted. A good computational solution should therefore be not only correct in code, but also statistically meaningful, numerically stable, and reproducible.”

A computational approach is a systematic way to solve a statistical problem when an exact analytical solution is difficult, unavailable, or inefficient. Examples include using Monte Carlo methods to approximate expectations, numerical optimization to estimate parameters, bootstrap methods to approximate sampling distributions, and cross-validation to select tuning parameters. In this course, the emphasis is on understanding the connection

Theory $\iff$ Computation $\iff$ Practice.

There is a general principle that will be repeated in this chapter that Kenneth Lange calls *optimization transfer* in his 1999 paper. The basic idea applies to the problem of maximizing a function f .

1. Direct optimize the function f .
 - It can be difficult
2. Optimize a surrogate function g that is easier to optimize than f .
3. So here, instead of optimize f , we optimize g .

Note 1: steps 2&3 are repeated until convergence.

Note 2: maximizing f is equivalent to minimizing $-f$.

Note 3: the surrogate function g should be chosen such that it is easier to optimize than f .

For instance, for a linear regression

$$y = X\beta + \varepsilon. \tag{2.1}$$

From regression class, we know that the (ordinary) least-squares estimation (OLE) for β is given by $\hat{\beta} = (X^\top X)^{-1} X^\top y$. It is convenient as the solution is in the **closed-form**! However, in the most case, the closed-form solutions will not be available.

For GLMs or non-linear regression, we need to do this **iteratively**!

2.1 Theory versus Computation

One confusing aspect of statistical computing is that often there is a disconnect between what is printed in a statistical computing textbook and what should be implemented on the computer.

- In textbooks, simpler to **present solutions as convenient mathematical formulas whenever possible**, in order to communicate basic ideas and to provide some insight.
 - However, directly translating these formulas into computer code is usually not advisable because there are many problematic aspects of computers that are simply not relevant when writing things down on paper.

Some potential issues include:

1. Memory overflow: The computer has a limited amount of memory, and it is possible to run out of memory when working with large datasets or complex models.
2. Numerical Precision: Sometimes, due to the cut precision of floating-point arithmetic, calculations that are mathematically equivalent can yield different results on a computer.
 - Example 1: round $1/3$ to two decimal places, we get 0.33. Then, $3 \cdot (1/3)$ is exactly 1, but $3 \cdot 0.33$ is 0.99.
 - Example 2: $1 - 0.99999999$ is 0.00000001 ($=1\text{E-}8$), but if we round 0.99999999 to two decimal places, we get 1.00, and then $1 - 1.00$ is 0. If we round 0.00000001 to two decimal places, we get 0.00.
 - Example 3: π
3. (Lienar) Dependence: The detection of linear dependence in matrix computations is influenced by machine precision. Since computers operate with finite precision, situations often arise where true linear dependence exists, but the computer cannot distinguish it from independence.
 - Example: Consider the matrix

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$$

The 3rd column is a linear combination of the first two columns (i.e., $\text{col3} = \text{col1} + \text{col2}$). However, due to machine precision limitations, the computer might not recognize this exact linear dependence, leading to numerical instability in computations involving this matrix. With a small distortion, we have

$$B = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 + 10^{-5} \end{pmatrix}$$

```
A <- matrix(
  c(1, 2, 3,
    4, 5, 6,
    7, 8, 9),
  nrow = 3, ncol = 3, byrow = TRUE)
B <- A
B[3, 3] <- B[3, 3] + 1E-5

qr(A)$rank
```

```
[1] 2
```

```
qr(B)$rank
```

```
[1] 3
```

2.2 Matrix Inversion

In many statistical analyses, such as linear regression and specify the distribution (such as normal distribution), matrix inversion plays a central role.

Example 2.1 (Normal distribution). We know that, a normal density with the parameters mean μ and standard deviation σ is

$$f(x | \mu, \sigma^2) = \frac{1}{\sqrt{2\pi}\sigma} \exp \left\{ -\frac{1}{2\sigma^2} (x - \mu)^2 \right\}$$

or we may work on the multivariate normal distribution case which is a bit more involved.

$X = (X_1, \dots, X_d)$ is said to be a multivariate normal distribution if and only if it is a linear combination of independent and identically distributed standard normals:

$$X = CZ + \mu, \quad Z = (Z_1, \dots, Z_d), \quad Z_i \stackrel{iid}{\sim} N(0, 1).$$

The property of the multivariate normal are:

- mean vector: $E(X) = \mu$
- variance: $Var(X) = CZC^\top = Cvar(Z)C^\top := \Sigma$

Notation: $X \sim N_d(\mu, \Sigma)$.

PDF:

$$f(x \mid \mu, \Sigma) = (2\pi)^{-d/2} \cdot \exp \left\{ -\frac{1}{2}(x - \mu)' \Sigma^{-1}(x - \mu) - \frac{1}{2} \log |\Sigma| \right\}.$$

Some of the potential ways to do this is to take logarithm of the PDF (Think about why).

Recall the linear regression model . The OLE for β is given by $\hat{\beta} = (X^\top X)^{-1} X^\top y$.

We can solve this using the R command

```
beta_hat <- solve(t(X) %*% X) %*% t(X) %*% y
```

where `solve()` is the R function for matrix inversion. However, it is not a desired way (think about why).

A better way is to go back to the formula, and look at

$$X^\top X \beta = X^\top y,$$

and solve this using the R command

```
solve( crossprod(X), crossprod(X, y) )
# this is the same as
# solve(t(X) %*% X, t(X) %*% y)
```

Here, we avoid explicitly calculating the inverse of $X^\top X$. Instead, we use gaussian elimination to solve the system of equations, which is generally more numerically stable and efficient.

2.2.0.1 Speed comparison

```
set.seed(2025-09-03)
X <- matrix(rnorm(5000 * 100), 5000, 100)
y <- rnorm(5000)
library(microbenchmark)
microbenchmark(solve(t(X) %*% X) %*% t(X) %*% y)
```

Unit: milliseconds

| | | | expr | min | lq |
|----------|----------|----------|---|----------|----------|
| | | | <code>solve(t(X) %*% X) %*% t(X) %*% y</code> | 28.83505 | 30.16593 |
| mean | median | uq | max | neval | |
| 31.96782 | 30.79489 | 32.63315 | 111.0151 | 100 | |

Warning message:

In `microbenchmark(solve(t(X) %*% X) %*% t(X) %*% y) :`
less accurate nanosecond times to avoid potential integer overflows

```
microbenchmark(solve(t(X) %*% X) %*% t(X) %*% y,  
               solve(crossprod(X), crossprod(X, y)))
```

Unit: milliseconds

| | | | expr | min | lq |
|----------|----------|----------|---|----------|----------|
| | | | <code>solve(t(X) %*% X) %*% t(X) %*% y</code> | 28.90135 | 30.11608 |
| | | | <code>solve(crossprod(X), crossprod(X, y))</code> | 25.05859 | 25.27480 |
| mean | median | uq | max | neval | |
| 31.78686 | 31.38513 | 32.66482 | 53.03354 | 100 | |
| 26.15771 | 25.81678 | 26.89188 | 29.12045 | 100 | |

Take home message 1

The take home here is that the issues arise from the finite precision of computer arithmetic and the limited memory available on computers. When implementing statistical methods on a computer, it is crucial to consider these limitations and choose algorithms and implementations that are robust to numerical issues.

2.2.1 Multi-collinearity

The above approach may break down when there is any multi-collinearity in the X matrix. For example, we can tack on a column to X that is very similar (but not identical) to the first column of X .

```
set.seed(7777)  
N <- 3000  
K <- 100  
y <- rnorm(N)  
X <- matrix(rnorm(N * K), N, K)  
W <- cbind(X, X[, 1] + rnorm(N, sd = 1E-15))
```



```
solve(crossprod(W), crossprod(W, y))

Error in `solve.default()`:
! system is computationally singular: reciprocal condition number = 1.36748e-32
```

The algorithm does not work because the cross product matrix $W^\top W$ is **singular**. In practice, matrices like these can come up a lot in data analysis and it would be useful to have a way to deal with it automatically.

R takes a different approach to solving for the unknown coefficients in a linear model. R uses the QR decomposition, which is not as fast, but has the added benefit of being able to automatically detect and handle colinear columns in the matrix.

Here, we use the fact that X can be decomposed as $X = QR$, where Q is an orthonormal matrix and R is an upper triangular matrix. Given that, we can rewrite $X^\top X\beta = X^\top y$ as

$$\begin{aligned} R^\top Q^\top QR\beta &= R^\top Q^\top y \\ R^\top IR\beta &= R^\top Q^\top y \\ R^\top R\beta &= R^\top Q^\top y, \end{aligned}$$

this leads to $R\beta = Q^\top y$. Now we can perform the Gaussian elimination to do it. Because R is an upper triangular matrix, the computational speed is much faster. Here, we **avoid to compute the cross product** $X^\top X$, which is numerical unstable if it is not *standardized* properly

We can see in R code that even with our singular matrix W above, the QR decomposition continues without error.

```
Qw <- qr(W)
str(Qw)
```

```
List of 4
 $ qr   : num [1:3000, 1:101] 54.43933 0.00123 -0.02004 -0.00671 -0.00178 ...
 $ rank : int 100
 $ graux: num [1:101] 1.01 1.01 1.01 1 1 ...
 $ pivot: int [1:101] 1 2 3 4 5 6 7 8 9 10 ...
 - attr(*, "class")= chr "qr"
```

Note that the output of `qr()` computes the rank of W to be 100, not 101 as the last column is collinear to the 1st column. From there, we can get $\hat{\beta}$ if we want using `qr.coef()`,

```
betahat <- qr.coef(Qw, y)
head(betahat, 3)
```

```
[1] 0.024314718 0.000916951 -0.005980588
```

```
tail(betahat, 3)
```

```
[1] 0.01545039 -0.01010440 NA
```

Q: Why there is an NA?

2.2.2 Trade-off

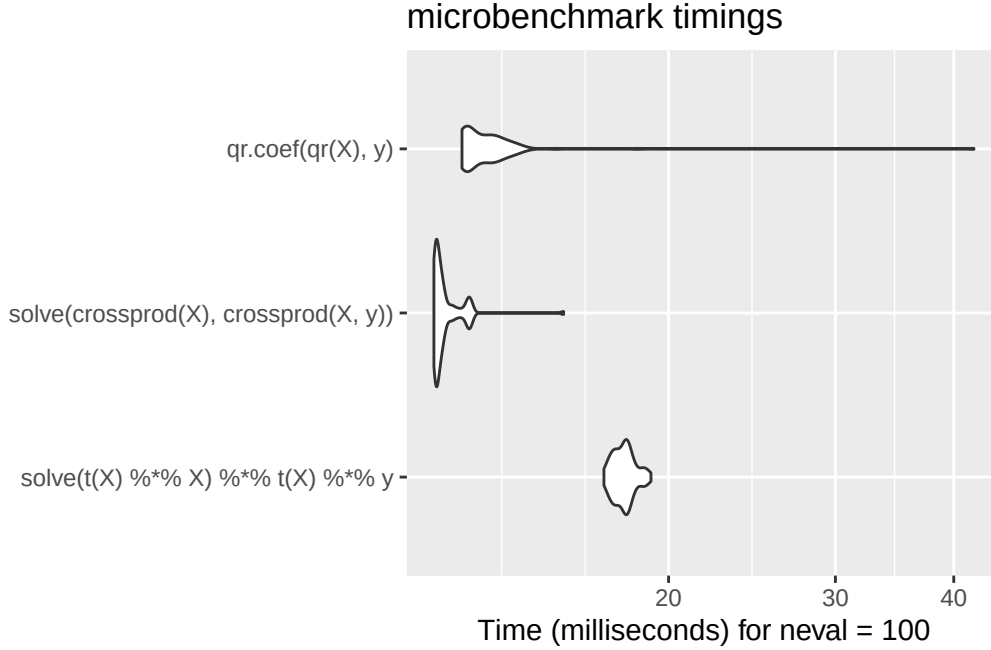
There isn't always elegance and flourish. When we take the robust approach, we accept that it comes at a cost.

```
library(ggplot2)
library(microbenchmark)
m <- microbenchmark(solve(t(X) %*% X) %*% t(X) %*% y,
                    solve(crossprod(X), crossprod(X, y)),
                    qr.coef(qr(X), y))
```

```
Warning in microbenchmark(solve(t(X) %*% X) %*% t(X) %*% y, solve(crossprod(X),
: less accurate nanosecond times to avoid potential integer overflows
```

```
autoplot(m)
```

```
Warning: `aes_string()` was deprecated in ggplot2 3.0.0.
i Please use tidy evaluation idioms with `aes()`.
i See also `vignette("ggplot2-in-packages")` for more information.
i The deprecated feature was likely used in the microbenchmark package.
  Please report the issue at
  <https://github.com/joshuaulrich/microbenchmark/issues/>.
```



Compared with the approaches discussed above, this method performs similarly to the naive approach but is much more stable and reliable.

In practice, we rarely call functions such as `qr()` or `qr.coef()` directly, since higher-level functions like `lm()` handle these computations automatically. However, in certain specialized and performance-critical settings, it can be advantageous to use alternative matrix decompositions to compute regression coefficients, especially when the computation must be repeated many times in a loop (i.e., *Vectorization*)

Example 2.2 (Multivariate Normal revisit). Computing the multivariate normal (MVN) density is a common task, for example, when fitting spatial models or Gaussian process models. Because maximum likelihood estimation (MLE) and likelihood ratio tests (LRT) often require evaluating the likelihood many times, efficiency is crucial.

After taking the log of the MVN density, we have

$$\ell(x \mid \mu, \Sigma) := \log \{f(x \mid \mu, \Sigma)\} = -\frac{d}{2} \log(2\pi) - \frac{1}{2} \log |\Sigma| - \frac{1}{2} (x - \mu)^\top \Sigma^{-1} (x - \mu).$$

On the right hand side, the first term is a constant, the second term is linear, and the last term is quadratic, which requires much more computational power.

A Naive Implementation

We first center the data $z := x - \mu$. Then we have $z^\top \Sigma^{-1} z$. This simplified the question for a bit.

Here, much like the linear regression example above, the key bottleneck is the inversion of the p -dimensional covariance matrix Σ . If we take z to be a $p \times 1$ column vector, then a literal translation of the mathematics into R code might look something like this,

```
t(z) %*% solve(Sigma) %*% z
```

To illustrate, let's simulate some data and compute the quadratic form the naive way:

```
set.seed(2025-09-03)

# Generate data
z <- matrix(rnorm(200 * 100), 200, 100)
S <- cov(z)

# Naive quadratic form
quad.naive <- function(z, S) {
  Sinv <- solve(S)
  rowSums((z %*% Sinv) * z)
}

library(dplyr)
quad.naive(z, S) %>% summary()
```

| Min. | 1st Qu. | Median | Mean | 3rd Qu. | Max. |
|-------|---------|--------|--------|---------|--------|
| 70.67 | 93.61 | 99.94 | 100.54 | 107.31 | 126.73 |

A Better Way: Cholesky Decomposition

Because the covariance matrix is symmetric and positive definite, we can exploit its **Cholesky decomposition**. That is, we write $\Sigma = R^\top R$, where R is a upper triangular matrix. Then,

$$z^\top \Sigma^{-1} z = z^\top (R^\top R)^{-1} z = z^\top R^{-1} R^{-\top} z = (R^{-\top} z)^\top (R^{-\top} z) := v^\top v.$$

Note that $v \in \mathbb{R}^p$ is the solution to the linear system $R^\top v = z$. Because R is upper triangular, we can solve this system efficiently using back substitution. Also, we can solve this without doing the inversion.

Once we have v we can compute its quadratic form $v^\top v$ by the `crossprod()` function.

```
quad.chol <- function(z, S) {
  R <- chol(S)
  v <- backsolve(R, t(z), transpose = TRUE)
```

```
colSums(v * v)
}

quad.chol(z, S) %>% summary()
```

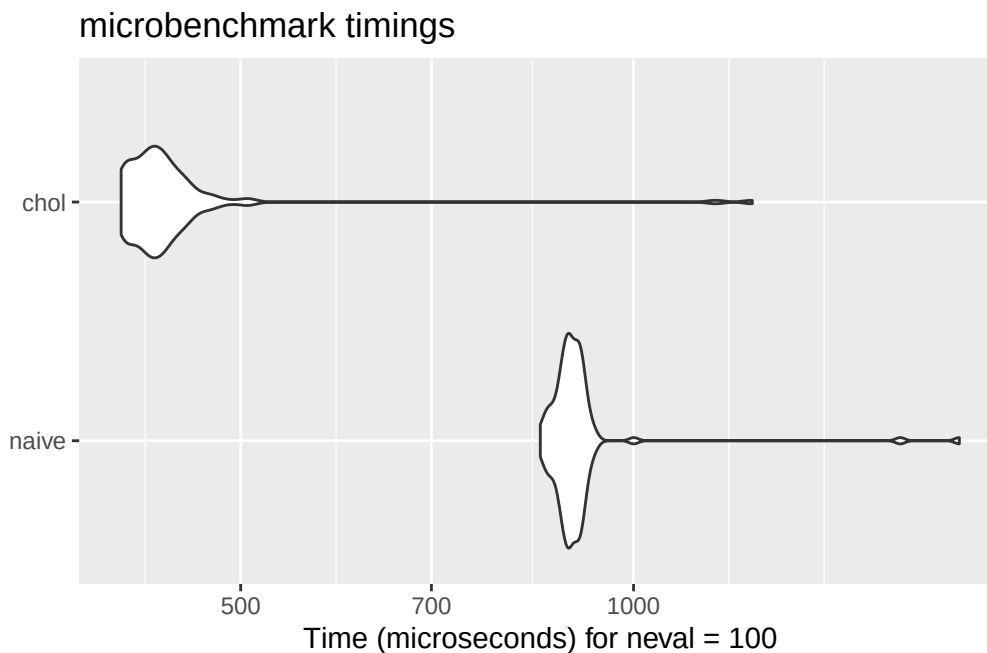
| Min. | 1st Qu. | Median | Mean | 3rd Qu. | Max. |
|-------|---------|--------|--------|---------|--------|
| 70.67 | 93.61 | 99.94 | 100.54 | 107.31 | 126.73 |

By product

Another benefit of the Cholesky decomposition is that it gives us a simple way to compute the log-determinant of Σ . The log-determinant of Σ is simply two times the sum of the log of the diagonal elements of R . (Why?)

Performance comparison

```
library(microbenchmark)
library(ggplot2)
m2 <- microbenchmark(
  naive = quad.naive(z, S),
  chol = quad.chol(z, S)
)
autoplot(m2)
```



Q: Why one is faster than the other?

💡 Take home message 2

The naive algorithm simply inverts the covariance matrix. The Cholesky-based approach, on the other hand, exploits the fact that covariance matrices are symmetric and positive definite. This results in an implementation that is both faster and numerically more stable—exactly the kind of optimization that makes a difference in real-world statistical computing.

Thus, a knowledge of statistics and numerical analysis can often lead to better algorithms, often invaluable!

Examples are borrowed from the following sources:

- Peng, R.D. [Advanced Statistical Computing](#).

3 Optimization

```
library(ggplot2)

theme_set(
  theme_minimal(base_size = 14)
)
```

Optimization plays an important role in statistical computing. Many statistical estimators can be written as

$$\hat{\theta} = \arg \min_{\theta} Q(\theta) \quad \text{or} \quad \hat{\theta} = \arg \max_{\theta} \ell(\theta),$$

where $Q(\theta)$ is an objective function and $\ell(\theta)$ may be a likelihood.

For simple problems, a closed-form solution may exist. For many statistical models, however, the solution must be obtained numerically.

Examples include maximum likelihood estimation, nonlinear regression, mixture models, machine learning, smoothing, and regularization.

The central idea is

| |
|---|
| Statistical Problem $\Leftrightarrow$ Optimization Problem $\Leftrightarrow$ Numerical Algorithm $\Leftrightarrow$ R Implementation |
|---|

3.1 Why Numerical Optimization?

Consider

$$X_1, \dots, X_n \stackrel{iid}{\sim} N(\mu, 1).$$

From mathematical statistics,

$$\hat{\mu}_{MLE} = \bar{X}.$$

We already know the closed-form solution. However, we can also obtain the estimator computationally by minimizing the negative log-likelihood.

```
set.seed(8670)

x <- rnorm(
  n = 50,
  mean = 2,
  sd = 1
)

nll <- function(mu) {
  -sum(
    dnorm(
      x,
      mean = mu,
      sd = 1,
      log = TRUE
    )
  )
}

fit <- optimize(
  nll,
  interval = c(-1, 5)
)

c(
  numerical = fit$minimum,
  theoretical = mean(x)
)
```

```
numerical theoretical
1.89689      1.89689
```

3.1.1 Visualizing the Objective Function

Rather than treating an optimizer as a black box, we can evaluate the objective function over a grid.

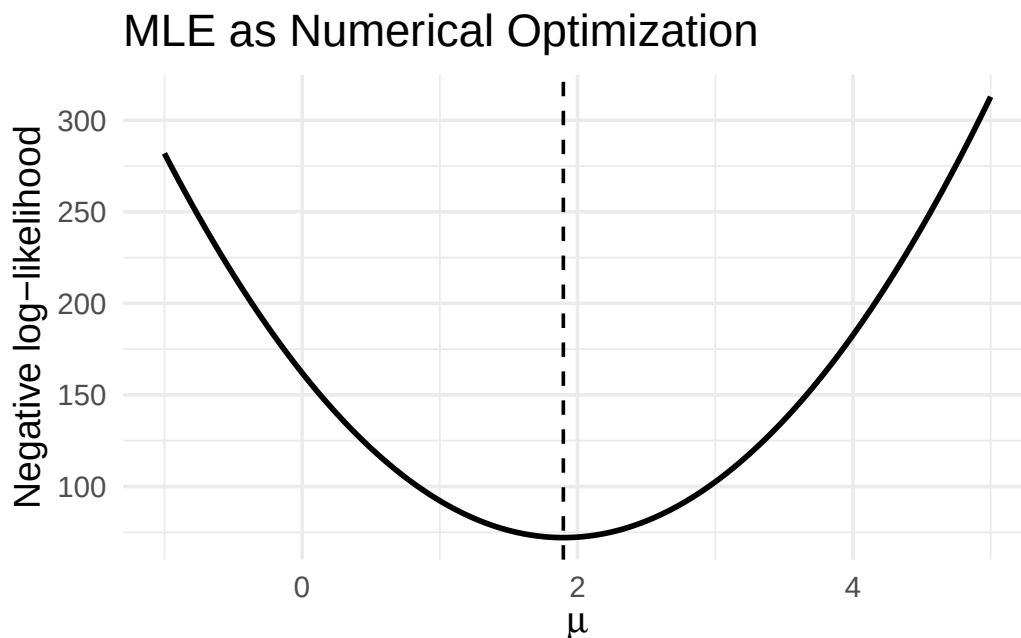

```

mu_grid <- seq(
  -1,
  5,
  length.out = 400
)

nll_data <- data.frame(
  mu = mu_grid,
  nll = vapply(
    mu_grid,
    nll,
    numeric(1)
  )
)

ggplot(
  nll_data,
  aes(
    x = mu,
    y = nll
  )
) +
  geom_line(linewidth = 1) +
  geom_vline(
    xintercept = mean(x),
    linetype = "dashed"
  ) +
  labs(
    x = expression(mu),
    y = "Negative log-likelihood",
    title = "MLE as Numerical Optimization"
  )

```



Why is numerical optimization useful here if we already know that

$$\hat{\mu}_{MLE} = \bar{X}?$$

The closed-form solution gives us a way to **validate the numerical algorithm**.

When a closed-form solution is unavailable, numerical optimization becomes essential.

3.2 When There Is No Closed-Form Solution

Consider the nonlinear regression model

$$Y_i = \exp(\theta X_i) + \varepsilon_i.$$

Suppose we estimate θ by minimizing

$$Q(\theta) = \sum_{i=1}^n [Y_i - \exp(\theta X_i)]^2.$$

Unlike ordinary linear regression, there is no convenient closed-form least-squares estimator.

```

set.seed(8670)

x <- seq(
  0,
  1,
  length.out = 50
)

y <- exp(1.5 * x) +
  rnorm(
    50,
    sd = 0.15
  )

sse <- function(theta) {
  sum(
    (y - exp(theta * x))^2
  )
}

fit_nonlinear <- optimize(
  sse,
  interval = c(0, 3)
)

fit_nonlinear$minimum

```

```
[1] 1.486825
```

```

theta_grid <- seq(
  0,
  3,
  length.out = 400
)

sse_data <- data.frame(
  theta = theta_grid,
  SSE = vapply(
    theta_grid,
    sse,
    numeric(1)
  )
)

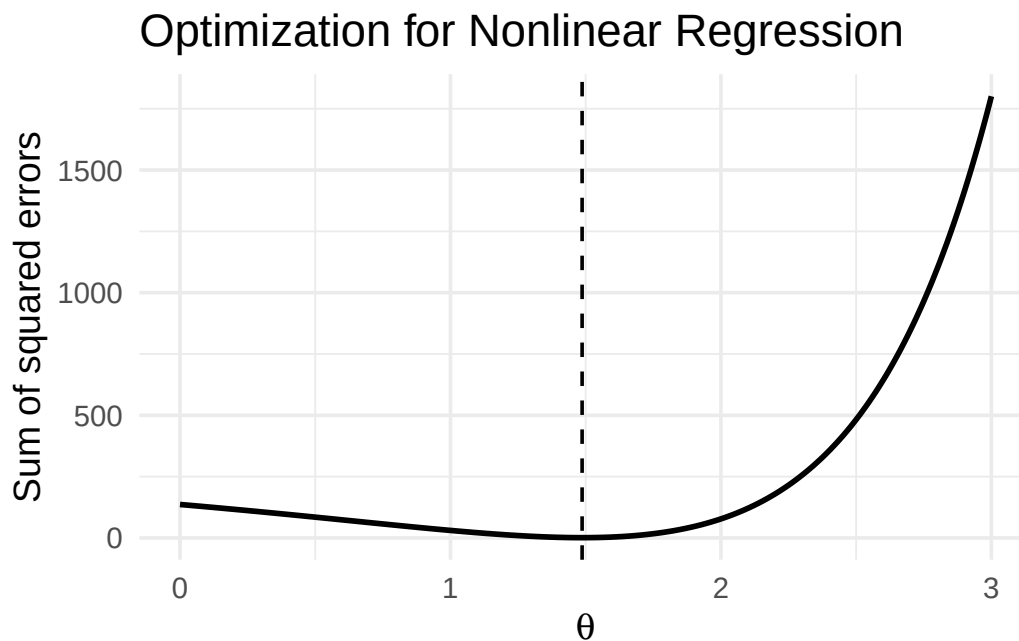
```

```

)
)

ggplot(
  sse_data,
  aes(
    x = theta,
    y = SSE
  )
) +
  geom_line(linewidth = 1) +
  geom_vline(
    xintercept = fit_nonlinear$minimum,
    linetype = "dashed"
  ) +
  labs(
    x = expression(theta),
    y = "Sum of squared errors",
    title = "Optimization for Nonlinear Regression"
  )
)

```



The important distinction is

Theory tells us what to optimize; computation tells us how to find it.

3.3 Two Families of Optimization Methods

Computational optimization methods can be broadly divided into two families:

Deterministic Methods and Metaheuristic Methods.

3.3.1 Deterministic Methods

A deterministic algorithm follows a predictable sequence of updates once the starting value is fixed.

Examples include:

- Newton's method;
- gradient descent;
- quasi-Newton methods such as BFGS;
- deterministic root-finding methods.

Advantages:

- usually efficient for smooth and well-behaved problems;
- mathematically well understood;
- convergence can often be studied theoretically;
- reproducible for a fixed starting value.

Limitations:

- may converge to a local rather than global optimum;
- can be sensitive to the starting value;
- some methods require derivatives;
- poor scaling can substantially slow convergence.

3.3.2 Metaheuristic Methods

Metaheuristic methods introduce randomness or population-based search to explore the parameter space more broadly.

Examples include:

- simulated annealing;
- genetic algorithms;
- particle swarm optimization.

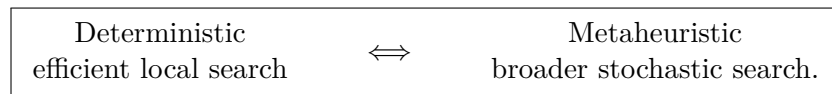
Advantages:

- can escape local optima;
- usually do not require derivatives;
- useful for multimodal or irregular objectives;
- can be applied to complicated search spaces.

Limitations:

- often computationally expensive;
- results can vary between runs;
- tuning parameters can strongly affect performance;
- global optimality is generally not guaranteed.

The basic trade-off is



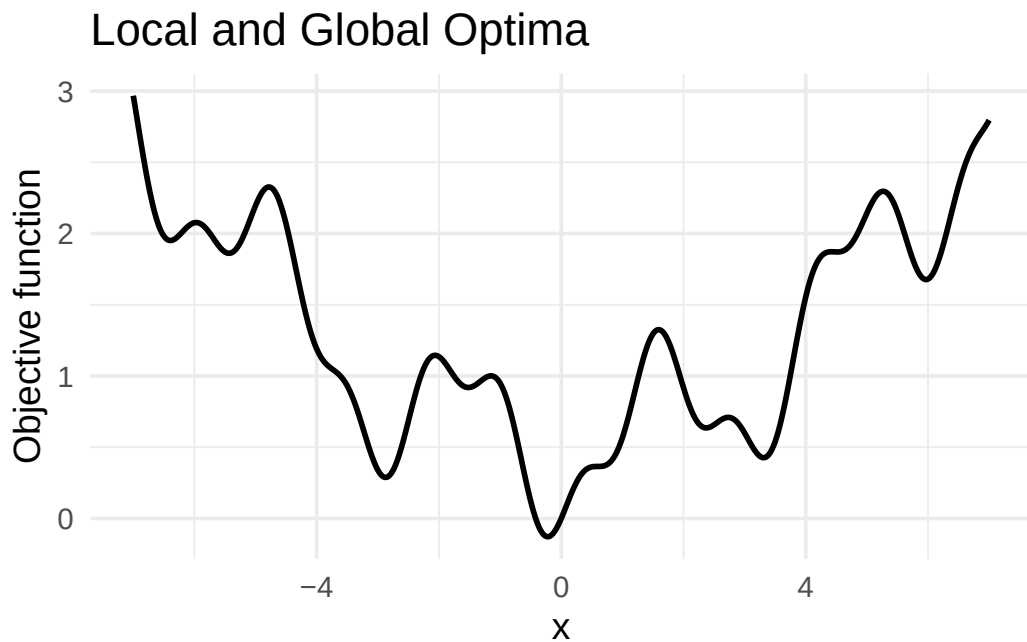
3.4 Local and Global Optima

Consider

$$Q(x) = 0.05x^2 + \sin^2(x) + 0.2 \sin(5x).$$

```
objective <- function(x) {  
  0.05 * x^2 +  
  sin(x)^2 +  
  0.2 * sin(5 * x)  
}  
  
x_grid <- seq(  
  -7,  
  7,  
  length.out = 1000  
)  
  
objective_data <- data.frame(  
  x = x_grid,  
  objective = objective(x_grid)  
)
```

```
ggplot(
  objective_data,
  aes(
    x = x,
    y = objective
  )
) +
  geom_line(linewidth = 1) +
  labs(
    x = "x",
    y = "Objective function",
    title = "Local and Global Optima"
  )
)
```



The objective contains several local minima.

A deterministic local optimizer may converge to different solutions depending on its starting value.

```
starts <- c(
  -6,
  -2,
  2,
  6
)
```

```

)

solutions <- vapply(
  starts,
  function(x0) {
    optim(
      par = x0,
      fn = objective,
      method = "BFGS"
    )$par
  },
  numeric(1)
)

local_results <- data.frame(
  start = starts,
  solution = solutions,
  objective = objective(solutions)
)

local_results

```

```

      start  solution objective
1      -6 -6.392105  1.9511520
2      -2 -1.519268  0.9193572
3       2  3.310780  0.4266970
4       6  5.967669  1.6769489

```

```

ggplot(
  objective_data,
  aes(
    x = x,
    y = objective
  )
) +
  geom_line(linewidth = 1) +
  geom_point(
    data = local_results,
    col = "red",
    aes(
      x = solution,

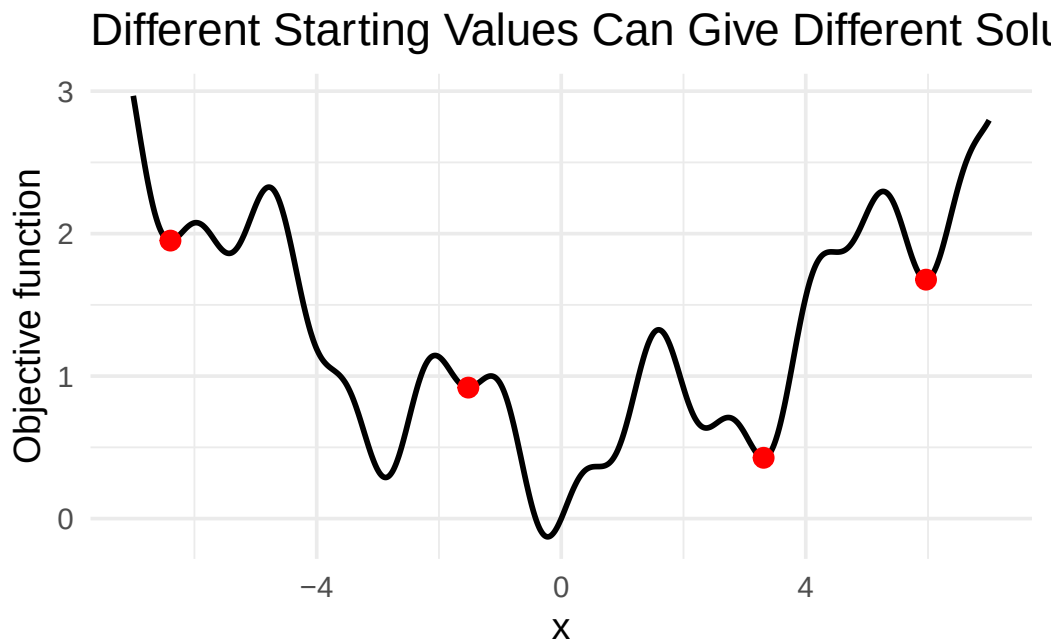
```



```

    y = objective
  ),
  size = 3
) +
labs(
  x = "x",
  y = "Objective function",
  title = "Different Starting Values Can Give Different Solutions"
)

```



If the algorithm is deterministic, why can different starting values still produce different answers?

The important distinction is

Local Optimum $\neq$ Global Optimum.

3.5 Deterministic Numerical Methods

We first study several deterministic numerical methods.

3.5.1 Root Finding

Root finding solves

$$f(x) = 0.$$

Why is root finding relevant to optimization?

If $Q(x)$ is differentiable and x^* is an interior optimum, then typically

$$Q'(x^*) = 0.$$

Therefore, optimization can sometimes be converted into

$$f(x) = Q'(x) = 0.$$

A root of $Q'(x)$ is only a candidate optimum. It may correspond to a minimum, maximum, or another stationary point.

3.5.2 A Statistical Example: Normal Quantiles

Suppose we want the 97.5th percentile of a standard normal distribution.

We seek z such that

$$P(Z \leq z) = 0.975.$$

Equivalently,

$$\Phi(z) - 0.975 = 0.$$

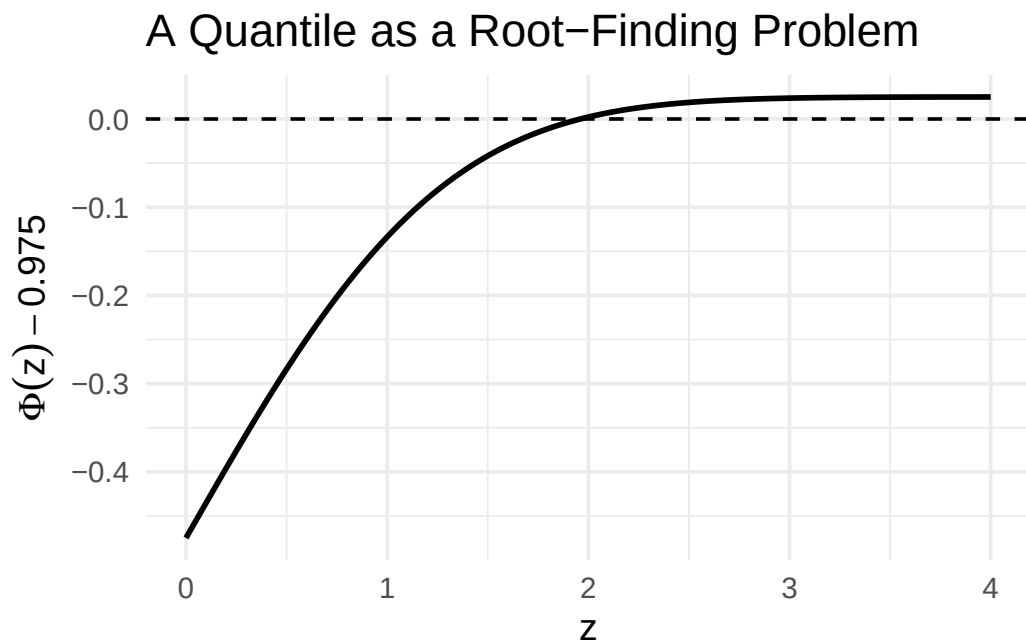
```
f_root <- function(z) {  
  pnorm(z) - 0.975  
}  
  
z_grid <- seq(  
  0,  
  4,  
  length.out = 400  
)
```

```

root_data <- data.frame(
  z = z_grid,
  f = f_root(z_grid)
)

ggplot(
  root_data,
  aes(
    x = z,
    y = f
  )
) +
  geom_line(linewidth = 1) +
  geom_hline(
    yintercept = 0,
    linetype = "dashed"
  ) +
  labs(
    x = "z",
    y = expression(Phi(z) - 0.975),
    title = "A Quantile as a Root-Finding Problem"
  )

```



R can solve the root directly:

```

root_R <- uniroot(
  f_root,
  interval = c(0, 4)
)$root

c(
  root_finding = root_R,
  qnorm = qnorm(0.975)
)

```

```

root_finding      qnorm
      1.959963      1.959964

```

Thus,

Quantile Calculation $\Leftrightarrow$ Root Finding.

Why do `uniroot()` and `qnorm()` give essentially the same answer?

3.5.3 Bisection Method

The bisection method is similar to a binary search.

Suppose f is continuous and

$$f(a)f(b) < 0.$$

Then there must be at least one root between a and b .

1. Choose a and b such that $f(a)f(b) < 0$.
2. Compute

$$c = \frac{a + b}{2}.$$

3. Keep the half containing the sign change.
4. Repeat until the interval is sufficiently small.

The main intuition is

Each iteration cuts the search interval approximately in half.

```
bisect_trace <- function(  
  f,  
  a,  
  b,  
  tol = 1e-6,  
  maxit = 100  
) {  
  
  if (f(a) * f(b) > 0) {  
    stop("f(a) and f(b) must have opposite signs.")  
  }  
  
  history <- data.frame()  
  
  for (k in 0:maxit) {  
  
    midpoint <- (a + b) / 2  
  
    history <- rbind(  
      history,  
      data.frame(  
        iteration = k,  
        a = a,  
        b = b,  
        midpoint = midpoint,  
        f_midpoint = f(midpoint)  
      )  
    )  
  
    if (  
      abs(f(midpoint)) < tol ||  
      (b - a) / 2 < tol  
    ) {  
      break  
    }  
  
    if (f(a) * f(midpoint) <= 0) {  
      b <- midpoint  
    }  
  }  
}
```

```

    } else {
      a <- midpoint
    }
  }

  history
}

bisect_history <- bisect_trace(
  f_root,
  a = 0,
  b = 4
)

tail(bisect_history)

```

| | iteration | a | b | midpoint | f_midpoint |
|----|-----------|----------|----------|----------|---------------|
| 7 | 6 | 1.937500 | 2.000000 | 1.968750 | 5.090967e-04 |
| 8 | 7 | 1.937500 | 1.968750 | 1.953125 | -4.023926e-04 |
| 9 | 8 | 1.953125 | 1.968750 | 1.960938 | 5.684292e-05 |
| 10 | 9 | 1.953125 | 1.960938 | 1.957031 | -1.718972e-04 |
| 11 | 10 | 1.957031 | 1.960938 | 1.958984 | -5.730834e-05 |
| 12 | 11 | 1.958984 | 1.960938 | 1.959961 | -1.780850e-07 |

3.5.4 Visualizing Bisection

```

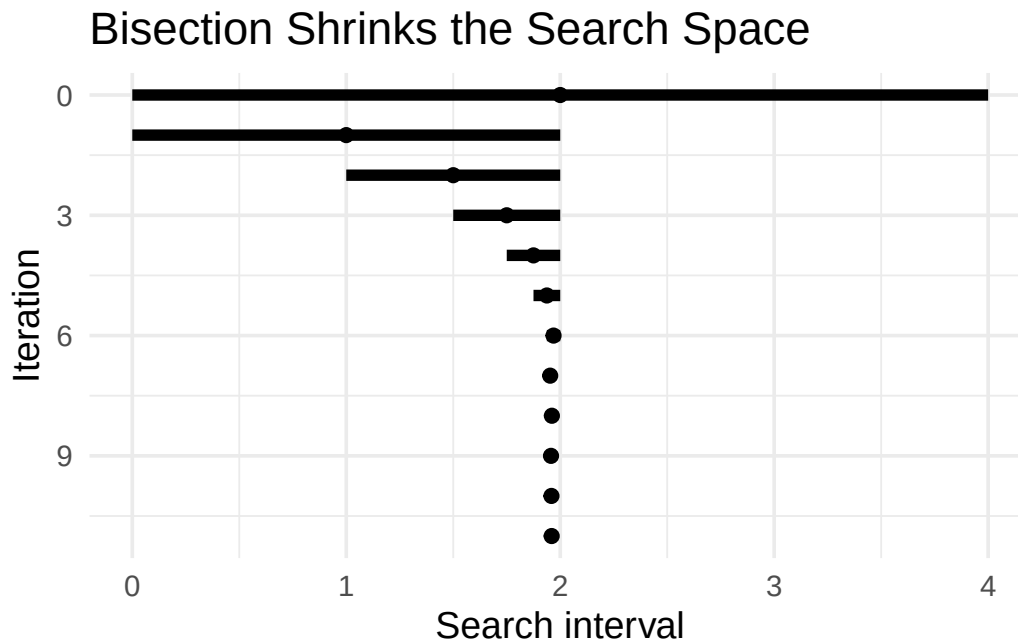
ggplot(
  bisect_history,
  aes(y = iteration)
) +
  geom_segment(
    aes(
      x = a,
      xend = b,
      yend = iteration
    ),
    linewidth = 2
  ) +
  geom_point(
    aes(x = midpoint),

```

```

    size = 2
) +
scale_y_reverse() +
labs(
  x = "Search interval",
  y = "Iteration",
  title = "Bisection Shrinks the Search Space"
)

```



Bisection requires continuity and a sign change, but does not require derivatives.

What information does bisection use about the function?

What information does it ignore?

3.5.5 Newton-Raphson Method

Newton's method uses local derivative information.

Around the current estimate x_k ,

$$f(x) \approx f(x_k) + f'(x_k)(x - x_k).$$

Set the approximation equal to zero:

$$0 = f(x_k) + f'(x_k)(x_{k+1} - x_k).$$

Therefore,

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}.$$

The intuition is:

Approximate the function locally by a straight line and use the root of that line as the next guess.

For

$$f(z) = \Phi(z) - 0.975,$$

we have

$$f'(z) = \phi(z).$$

```
f <- function(z) {
  pnorm(z) - 0.975
}

df <- function(z) {
  dnorm(z)
}
```

```
newton_trace <- function(
  f,
  df,
  x0,
  tol = 1e-8,
  maxit = 100
) {

  x <- x0

  history <- data.frame(
    iteration = 0,
```



```

    x = x,
    fx = f(x)
  )

  for (k in seq_len(maxit)) {

    dfx <- df(x)

    if (abs(dfx) < 1e-12) {
      stop("Derivative is too close to zero.")
    }

    x_new <- x - f(x) / dfx

    history <- rbind(
      history,
      data.frame(
        iteration = k,
        x = x_new,
        fx = f(x_new)
      )
    )

    if (abs(x_new - x) < tol) {
      break
    }

    x <- x_new
  }

  history
}

newton_history <- newton_trace(
  f,
  df,
  x0 = 1
)

newton_history

```

| iteration | x | fx |
|-----------|---|----|
|-----------|---|----|

| | | | |
|---|---|----------|---------------|
| 1 | 0 | 1.000000 | -1.336553e-01 |
| 2 | 1 | 1.552361 | -3.528790e-02 |
| 3 | 2 | 1.847484 | -7.338509e-03 |
| 4 | 3 | 1.948843 | -6.570601e-04 |
| 5 | 4 | 1.959844 | -7.006182e-06 |
| 6 | 5 | 1.959964 | -8.227717e-10 |
| 7 | 6 | 1.959964 | 0.000000e+00 |
| 8 | 7 | 1.959964 | 0.000000e+00 |

3.5.6 Visualizing Newton's Method

```
tangent_data <- data.frame(
  x = head(newton_history$x, -1),
  y = head(newton_history$fx, -1),
  xend = tail(newton_history$x, -1),
  yend = 0
)

z_grid <- seq(
  0.5,
  3,
  length.out = 400
)

newton_function_data <- data.frame(
  z = z_grid,
  f = f(z_grid)
)

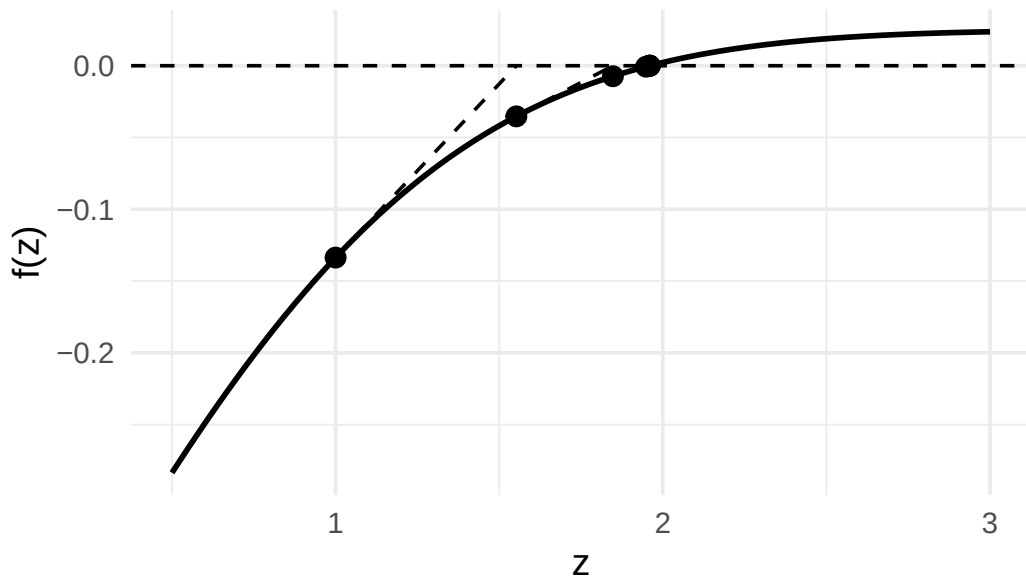
ggplot(
  newton_function_data,
  aes(
    x = z,
    y = f
  )
) +
  geom_line(linewidth = 1) +
  geom_hline(
    yintercept = 0,
    linetype = "dashed"
  ) +
```

```

geom_segment(
  data = tangent_data,
  aes(
    x = x,
    y = y,
    xend = xend,
    yend = yend
  ),
  inherit.aes = FALSE,
  linetype = "dashed"
) +
geom_point(
  data = newton_history,
  aes(
    x = x,
    y = fx
  ),
  inherit.aes = FALSE,
  size = 3
) +
labs(
  x = "z",
  y = "f(z)",
  title = "Newton's Method: Follow the Local Tangent"
)

```

Newton's Method: Follow the Local Tangent



Newton's method can converge very quickly near the root.

However, if

$$f'(x_k) \approx 0,$$

then the update may become very large and unstable.

What happens to Newton's update when $f'(x_k)$ is close to zero?

3.5.7 Secant Method

Newton's method requires the derivative $f'(x)$.

If the derivative is unavailable or expensive, we can approximate it using two previous points:

$$f'(x_k) \approx \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}.$$

Substituting this into Newton's update gives

$$x_{k+1} = x_k - f(x_k) \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}.$$

```

secant_trace <- function(
  f,
  x0,
  x1,
  tol = 1e-8,
  maxit = 100
) {

  history <- data.frame(
    iteration = c(0, 1),
    x = c(x0, x1),
    fx = c(
      f(x0),
      f(x1)
    )
  )

  for (k in 2:maxit) {

    f0 <- f(x0)
    f1 <- f(x1)

    denominator <- f1 - f0

    if (abs(denominator) < 1e-12) {
      stop("Secant denominator is too close to zero.")
    }

    x2 <- x1 -
      f1 * (x1 - x0) /
      denominator

    history <- rbind(
      history,
      data.frame(
        iteration = k,
        x = x2,
        fx = f(x2)
      )
    )

    if (abs(x2 - x1) < tol) {

```

```

        break
    }

    x0 <- x1
    x1 <- x2
}

history
}

secant_history <- secant_trace(
  f,
  x0 = 1,
  x1 = 2.5
)

secant_history

```

```

      iteration      x      fx
1           0 1.000000 -1.336553e-01
2           1 2.500000  1.879033e-02
3           2 2.315111  1.469657e-02
4           3 1.651361 -2.433241e-02
5           4 2.065173  5.546662e-03
6           5 1.988354  1.613717e-03
7           6 1.956835 -1.834592e-04
8           7 1.960052  5.151979e-06
9           8 1.959964  1.577731e-08
10          9 1.959964 -1.363021e-12
11         10 1.959964  0.000000e+00

```

The three methods illustrate different computational trade-offs.

| Method | Derivative? | Starting information | Main advantage | Main limitation |
|-----------|-------------|----------------------|----------------------------|-----------------|
| Bisection | No | $[a, b]$ | Very reliable | Slower |
| Newton | Yes | x_0 | Very fast
near root | Can fail |
| Secant | No | x_0, x_1 | Fast without
derivative | Less reliable |

3.6 Convergence

Suppose an iterative algorithm produces

$$x_0, x_1, x_2, \dots$$

and converges to a solution x^* .

Define

$$e_k = |x_k - x^*|.$$

A common description of convergence is

$$e_{k+1} \approx C e_k^q.$$

The value q describes how quickly the error decreases.

- **Linear convergence:** $q = 1$.
- **Superlinear convergence:** faster than linear.
- **Quadratic convergence:** $q = 2$.

Under suitable conditions:

- bisection converges linearly;
- the secant method converges superlinearly;
- Newton's method converges quadratically near the root.

3.6.1 Comparing Convergence in R

```
true_root <- qnorm(0.975)

bisection_error <- data.frame(
  iteration = bisect_history$iteration,
  error = abs(
    bisect_history$midpoint -
    true_root
  ),
  method = "Bisection"
)
```

```

newton_error <- data.frame(
  iteration = newton_history$iteration,
  error = abs(
    newton_history$x -
    true_root
  ),
  method = "Newton"
)

secant_error <- data.frame(
  iteration = secant_history$iteration,
  error = abs(
    secant_history$x -
    true_root
  ),
  method = "Secant"
)

convergence_data <- rbind(
  bisection_error,
  newton_error,
  secant_error
)

convergence_data <- subset(
  convergence_data,
  error > .Machine$double.eps
)

```

```

ggplot(
  convergence_data,
  aes(
    x = iteration,
    y = error,
    linetype = method,
    shape = method
  )
) +
  geom_line() +
  geom_point(size = 2) +
  scale_y_log10() +
  labs(

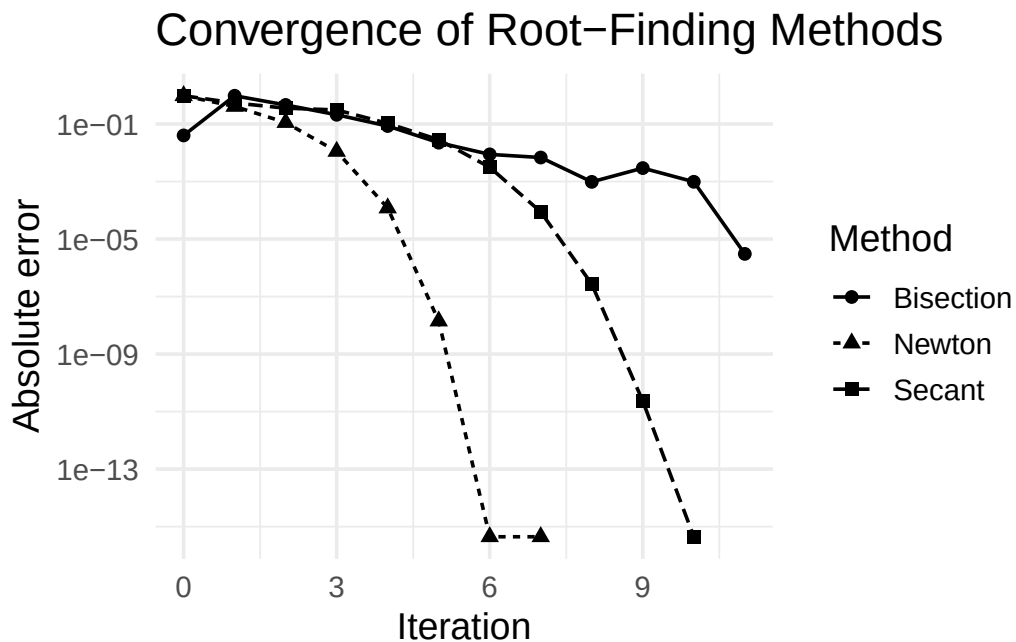
```



```

x = "Iteration",
y = "Absolute error",
title = "Convergence of Root-Finding Methods",
linetype = "Method",
shape = "Method"
)

```



Which method appears to converge fastest?

Does fewer iterations necessarily mean less computation?

A useful computational perspective is

Total Computational Cost $\approx$ Cost per Iteration $\times$ Number of Iterations.

3.7 From One Dimension to Multiple Dimensions

Most statistical problems involve more than one parameter.

Instead of

$$x \in \mathbb{R},$$

we now consider

$$\theta = (\theta_1, \dots, \theta_p)^\top \in \mathbb{R}^p.$$

The one-dimensional concepts generalize naturally:

$$Q'(x) \implies \nabla Q(\theta),$$

and

$$Q''(x) \implies H(\theta).$$

The **gradient** is

$$\nabla Q(\theta) = \begin{pmatrix} \partial Q / \partial \theta_1 \\ \vdots \\ \partial Q / \partial \theta_p \end{pmatrix},$$

and the **Hessian** is the matrix of second derivatives,

$$H(\theta) = \left[\frac{\partial^2 Q}{\partial \theta_j \partial \theta_k} \right].$$

3.7.1 Visualizing a Two-Dimensional Objective

Consider

$$Q(\theta_1, \theta_2) = (\theta_1 - 2)^2 + 4(\theta_2 + 1)^2.$$

```
objective_2d <- function(theta1, theta2) {  
  (theta1 - 2)^2 +  
    4 * (theta2 + 1)^2  
}  
  
theta1_grid <- seq(  
  -2,  
  6,  
  length.out = 150  
)
```

```

theta2_grid <- seq(
  -4,
  2,
  length.out = 150
)

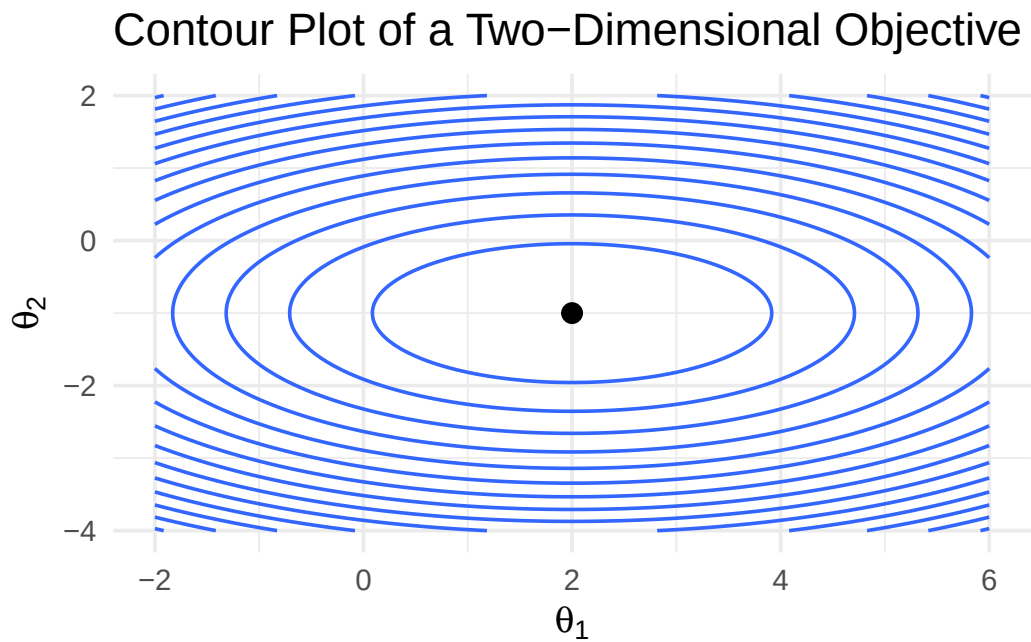
objective_grid <- expand_grid(
  theta1 = theta1_grid,
  theta2 = theta2_grid
)

objective_grid$value <- with(
  objective_grid,
  objective_2d(theta1, theta2)
)

ggplot(
  objective_grid,
  aes(
    x = theta1,
    y = theta2,
    z = value
  )
) +
  geom_contour(
    bins = 15
  ) +
  geom_point(
    data = data.frame(
      theta1 = 2,
      theta2 = -1
    ),
    aes(
      x = theta1,
      y = theta2
    ),
    inherit.aes = FALSE,
    size = 3
  ) +
  labs(
    x = expression(theta[1]),
    y = expression(theta[2]),

```

```
title = "Contour Plot of a Two-Dimensional Objective"
)
```



The minimum occurs at

$$\theta^* = \begin{pmatrix} 2 \\ -1 \end{pmatrix}.$$

3.8 Gradient Descent

The gradient points in the direction of greatest increase.

Therefore,

$$-\nabla Q(\theta)$$

is a natural direction for decreasing the objective.

Gradient descent updates

$$\theta_{k+1} = \theta_k - \alpha_k \nabla Q(\theta_k),$$

where $\alpha_k > 0$ is the **step size** or **learning rate**.

For the quadratic example,

$$\nabla Q(\theta_1, \theta_2) = \begin{pmatrix} 2(\theta_1 - 2) \\ 8(\theta_2 + 1) \end{pmatrix}.$$

```
gradient_2d <- function(theta) {  
  c(  
    2 * (theta[1] - 2),  
    8 * (theta[2] + 1)  
  )  
}  
  
gradient_descent <- function(  
  start,  
  alpha = 0.1,  
  maxit = 30  
) {  
  
  theta <- start  
  
  history <- data.frame(  
    iteration = 0,  
    theta1 = theta[1],  
    theta2 = theta[2]  
  )  
  
  for (k in seq_len(maxit)) {  
  
    theta <-  
      theta -  
      alpha * gradient_2d(theta)  
  
    history <- rbind(  
      history,  
      data.frame(  
        iteration = k,  
        theta1 = theta[1],  
        theta2 = theta[2]  
      )  
    )  
  }  
}
```

```

    history
  }

gd_path <- gradient_descent(
  start = c(-1, 1),
  alpha = 0.1,
  maxit = 20
)

```

3.8.1 Visualizing the Optimization Path

```

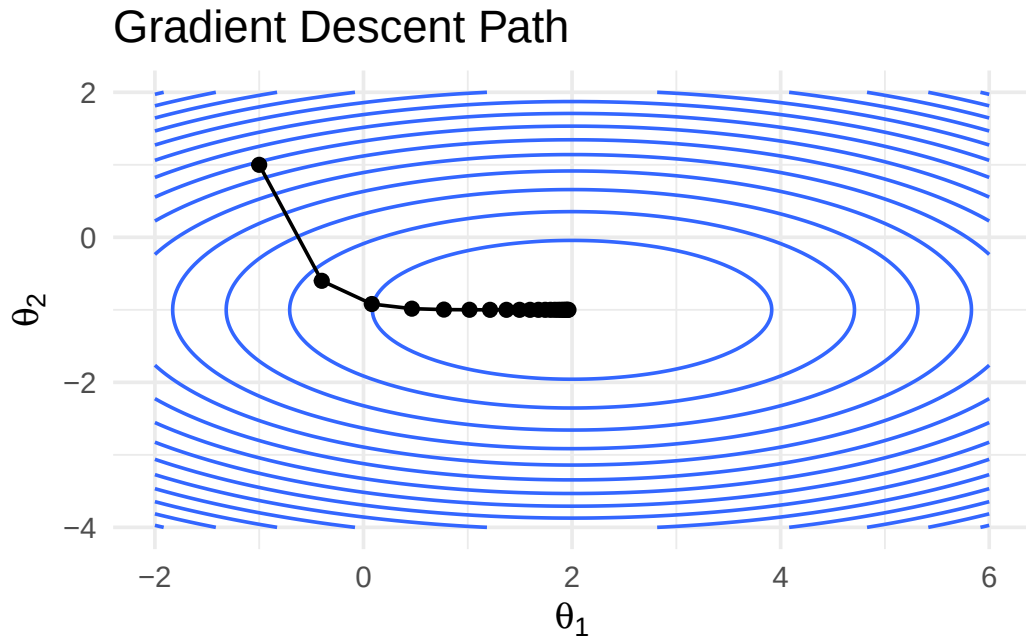
ggplot(
  objective_grid,
  aes(
    x = theta1,
    y = theta2,
    z = value
  )
) +
  geom_contour(
    bins = 15
  ) +
  geom_path(
    data = gd_path,
    aes(
      x = theta1,
      y = theta2
    ),
    inherit.aes = FALSE
  ) +
  geom_point(
    data = gd_path,
    aes(
      x = theta1,
      y = theta2
    ),
    inherit.aes = FALSE,
    size = 2
  ) +
  labs(

```

```

x = expression(theta[1]),
y = expression(theta[2]),
title = "Gradient Descent Path"
)

```



What happens if the step size α is too small?

What happens if it is too large?

3.8.2 Step Size Matters

```

gd_small <- gradient_descent(
  start = c(-1, 1),
  alpha = 0.02,
  maxit = 30
)

gd_medium <- gradient_descent(
  start = c(-1, 1),
  alpha = 0.1,
  maxit = 30
)

```

```

gd_large <- gradient_descent(
  start = c(-1, 1),
  alpha = 0.24,
  maxit = 30
)

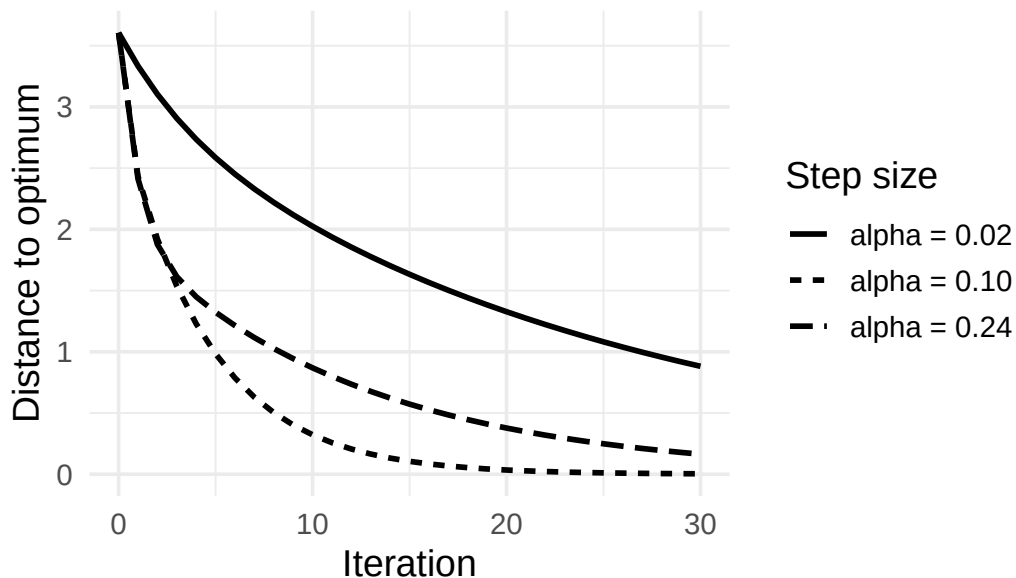
gd_compare <- rbind(
  transform(
    gd_small,
    step = "alpha = 0.02"
  ),
  transform(
    gd_medium,
    step = "alpha = 0.10"
  ),
  transform(
    gd_large,
    step = "alpha = 0.24"
  )
)

gd_compare$error <-
  sqrt(
    (gd_compare$theta1 - 2)^2 +
    (gd_compare$theta2 + 1)^2
  )

ggplot(
  gd_compare,
  aes(
    x = iteration,
    y = error,
    linetype = step
  )
) +
  geom_line(linewidth = 1) +
  labs(
    x = "Iteration",
    y = "Distance to optimum",
    linetype = "Step size",
    title = "Step Size Changes the Convergence Behaviour"
  )

```


Step Size Changes the Convergence Behaviour



3.9 Multivariate Newton's Method

In one dimension,

$$x_{k+1} = x_k - \frac{Q'(x_k)}{Q''(x_k)}.$$

In multiple dimensions, this becomes

$$\theta_{k+1} = \theta_k - H(\theta_k)^{-1} \nabla Q(\theta_k).$$

However, computationally we usually do **not** calculate the inverse explicitly.

Instead, solve

$$H(\theta_k) d_k = -\nabla Q(\theta_k),$$

and update

$$\theta_{k+1} = \theta_k + d_k.$$

Why do we solve a linear system instead of explicitly computing

$$H^{-1}?$$

This connects directly to the earlier discussion of numerical stability and matrix inversion.

3.10 Quasi-Newton Methods

Newton's method requires the Hessian.

For high-dimensional statistical models, computing the Hessian can be expensive.

Quasi-Newton methods approximate curvature information rather than calculating the Hessian directly.

One of the most widely used methods is **BFGS**.

In R:

```
optim(  
  par = start,  
  fn = objective,  
  method = "BFGS"  
)
```

The main computational idea is

Use curvature information without repeatedly computing the full Hessian.

3.11 Statistical Example: Logistic Regression

Consider

$$Y_i \sim \text{Bernoulli}(p_i),$$

with

$$\log\left(\frac{p_i}{1-p_i}\right) = \beta_0 + \beta_1 x_i.$$

The parameter is now a vector

$$\beta = (\beta_0, \beta_1)^\top.$$

There is no closed-form MLE, so logistic regression requires numerical optimization.

```
set.seed(8670)

n <- 200

x_logit <- rnorm(n)

beta_true <- c(
  -0.5,
  1.5
)

eta <-
  beta_true[1] +
  beta_true[2] * x_logit

p <- plogis(eta)

y_logit <- rbinom(
  n,
  size = 1,
  prob = p
)
```

Define the negative log-likelihood:

```
logistic_nll <- function(beta) {

  eta <-
    beta[1] +
    beta[2] * x_logit

  p <- plogis(eta)

  -sum(
    dbinom(
      y_logit,
```

```

        size = 1,
        prob = p,
        log = TRUE
    )
}

```

Estimate the coefficients using BFGS:

```

fit_optim <- optim(
  par = c(0, 0),
  fn = logistic_nll,
  method = "BFGS"
)

fit_optim$par

```

```
[1] -0.381730  1.014505
```

Compare with `glm()`:

```

fit_glm <- glm(
  y_logit ~ x_logit,
  family = binomial()
)

coef(fit_glm)

```

```

(Intercept)      x_logit
-0.3817297      1.0145052

```

```
fit_optim$par
```

```
[1] -0.381730  1.014505
```

Why is comparing `optim()` with `glm()` useful?

This illustrates the complete computational-statistics workflow:

| |
|---|
| Statistical Model → Likelihood → Optimization → Algorithm → Validation. |
|---|

3.12 Scaling and Conditioning

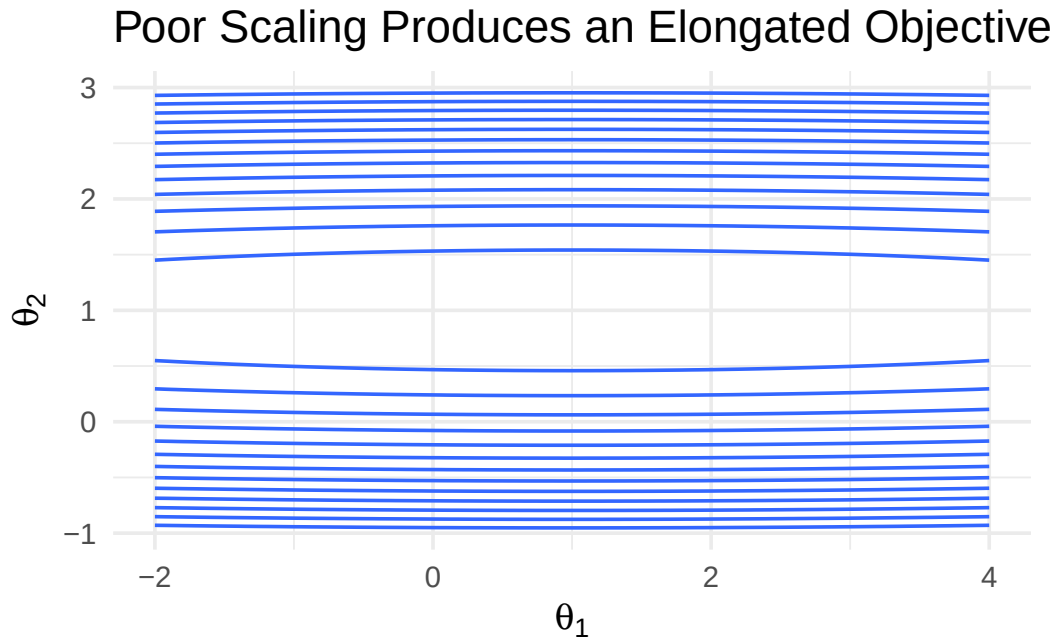
Optimization can behave very differently when parameters operate on different scales.

Consider

$$Q(\theta_1, \theta_2) = (\theta_1 - 1)^2 + 100(\theta_2 - 1)^2.$$

```
scaled_objective <- function(theta1, theta2) {  
  (theta1 - 1)^2 +  
    100 * (theta2 - 1)^2  
}  
  
scale_grid <- expand_grid(  
  theta1 = seq(  
    -2,  
    4,  
    length.out = 150  
  ),  
  theta2 = seq(  
    -1,  
    3,  
    length.out = 150  
  )  
)  
  
scale_grid$value <- with(  
  scale_grid,  
  scaled_objective(  
    theta1,  
    theta2  
  )  
)  
  
ggplot(  
  scale_grid,  
  aes(  
    x = theta1,  
    y = theta2,  
    z = value  
  )  
) +
```

```
geom_contour(  
  bins = 15  
) +  
labs(  
  x = expression(theta[1]),  
  y = expression(theta[2]),  
  title = "Poor Scaling Produces an Elongated Objective"  
)
```



A highly elongated objective can cause gradient-based algorithms to zig-zag and converge slowly.

This illustrates another important principle:

Same mathematical problem + different scaling $\Rightarrow$ different computational behaviour.

3.13 Metaheuristic Methods

Deterministic optimization methods can work extremely well for smooth and well-behaved objectives.

However, they may struggle when the objective:

- contains many local optima;
- is not differentiable;
- is discontinuous;
- has a complicated search space.

Metaheuristic methods use broader stochastic exploration.

The key idea is

Explore broadly $\longrightarrow$ search more locally later.

3.13.1 Why Accept a Worse Solution?

A local deterministic method usually attempts to improve the objective at every step.

A stochastic method may deliberately accept a worse solution.

Why would an optimization algorithm ever accept a worse solution?

Temporarily moving to a worse location can allow the algorithm to escape a local optimum and later reach a better region.

3.14 Simulated Annealing

Simulated annealing is a stochastic optimization method inspired by the physical process of heating and slowly cooling a material.

Suppose the current solution is x_k and a candidate solution x' is generated.

For a minimization problem, if

$$Q(x') < Q(x_k),$$

the new solution is accepted.

If

$$Q(x') > Q(x_k),$$

the worse solution may still be accepted with probability

$$P(\text{accept}) = \exp\left(-\frac{\Delta Q}{T}\right),$$

where

$$\Delta Q = Q(x') - Q(x_k) > 0.$$

The parameter T is called the **temperature**.

When

T is large,

the algorithm explores more broadly.

When

T is small,

the algorithm becomes more selective.

3.14.1 Visualizing the Acceptance Probability

```
delta_grid <- seq(
  0,
  5,
  length.out = 300
)

temperature <- c(
  0.1,
  0.5,
  2
)

sa_probability <- expand.grid(
  delta = delta_grid,
  temperature = temperature
)

sa_probability$probability <-
  exp(
    -sa_probability$delta /
```



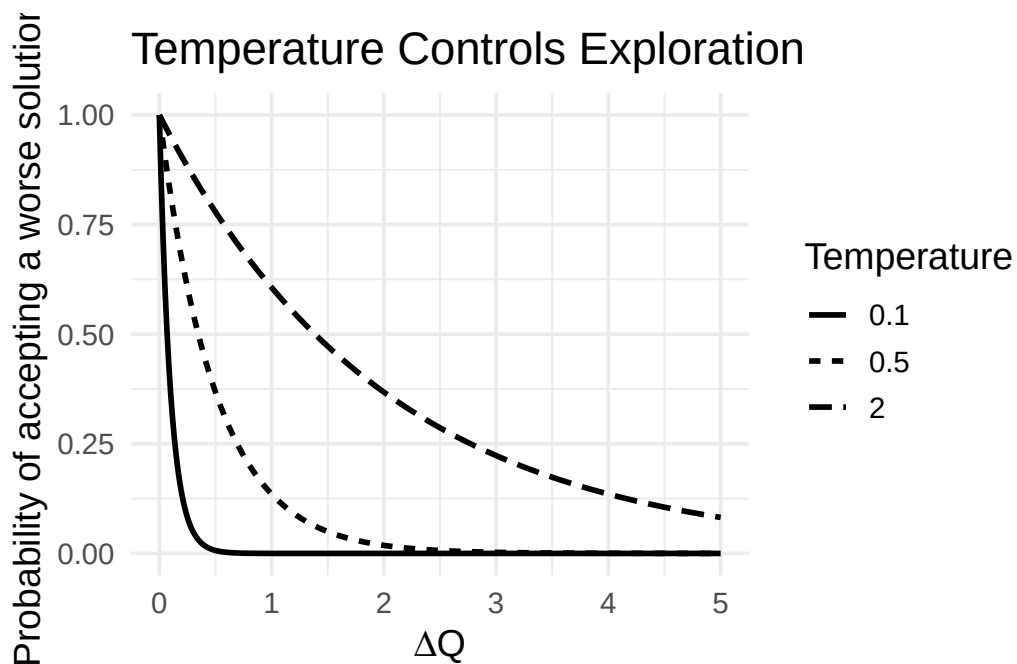
```

    sa_probability$temperature
  )

sa_probability$temperature <-
  factor(
    sa_probability$temperature
  )

ggplot(
  sa_probability,
  aes(
    x = delta,
    y = probability,
    linetype = temperature
  )
) +
  geom_line(linewidth = 1) +
  labs(
    x = expression(Delta * Q),
    y = "Probability of accepting a worse solution",
    linetype = "Temperature",
    title = "Temperature Controls Exploration"
  )

```



At high temperature, why is the algorithm more willing to accept a poor move?

What happens as the temperature approaches zero?

3.14.2 Simulated Annealing in R

R provides simulated annealing through

```
optim(  
  ...,  
  method = "SANN"  
)
```

Using the earlier multimodal objective:

```
set.seed(8670)  
  
sa_fit <- optim(  
  par = 5,  
  fn = objective,  
  method = "SANN",  
  control = list(  
    maxit = 3000  
  )  
)  
  
sa_fit$par
```

```
[1] -0.1958383
```

```
sa_fit$value
```

```
[1] -0.1262269
```

Because simulated annealing is stochastic, different seeds may produce different results.

```

seeds <- 1:20

sa_solution <- vapply(
  seeds,
  function(seed) {

    set.seed(seed)

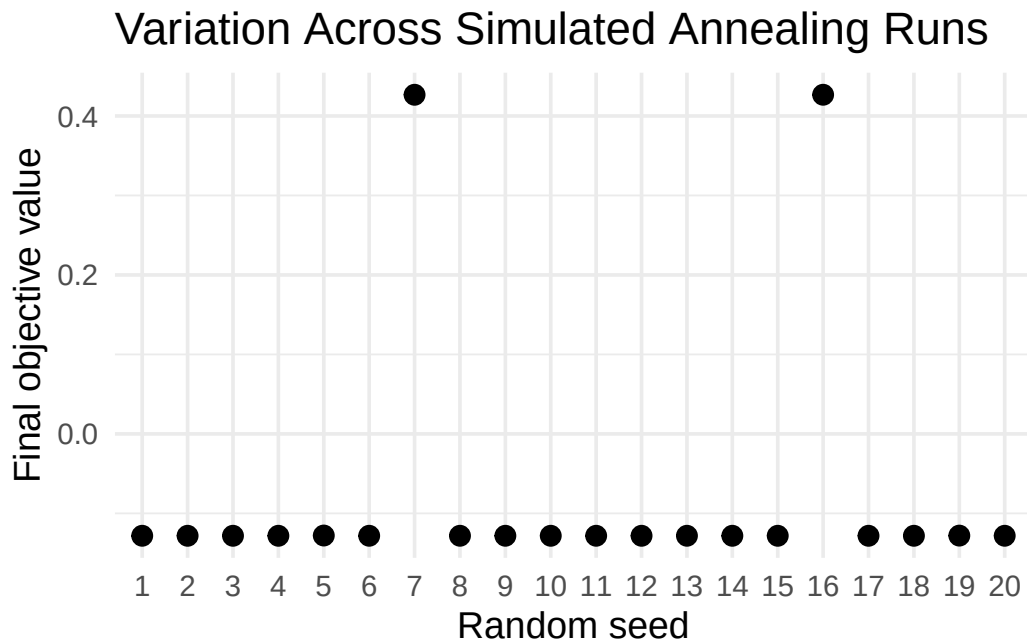
    optim(
      par = 5,
      fn = objective,
      method = "SANN",
      control = list(
        maxit = 3000
      )
    )$par

  },
  numeric(1)
)

sa_data <- data.frame(
  seed = seeds,
  solution = sa_solution,
  objective = objective(sa_solution)
)

ggplot(
  sa_data,
  aes(
    x = factor(seed),
    y = objective
  )
) +
  geom_point(size = 3) +
  labs(
    x = "Random seed",
    y = "Final objective value",
    title = "Variation Across Simulated Annealing Runs"
  )

```



3.14.3 Advantages of Simulated Annealing

- Can escape local optima.
- Does not require derivatives.
- Easy to adapt to many objective functions.
- Useful for multimodal problems.

3.14.4 Limitations of Simulated Annealing

- Can require many function evaluations.
- Results vary across runs.
- Performance depends on the cooling schedule.
- Does not guarantee the global optimum.

3.15 Genetic Algorithm

A genetic algorithm uses a **population of candidate solutions** rather than one current solution.

The basic cycle is

| |
|--|
| Selection → Crossover → Mutation → New Generation. |
|--|

3.15.1 Selection

Better candidate solutions are more likely to remain in the population.

3.15.2 Crossover

Information from two candidate solutions is combined to create new candidates.

3.15.3 Mutation

Random changes are introduced to preserve diversity and explore new regions.

The key difference from simulated annealing is:

| | |
|---------------------------|--------------------------------|
| SA: one evolving solution | GA: a population of solutions. |
|---------------------------|--------------------------------|

3.15.4 Advantages

- Explores several regions simultaneously.
- Does not require gradients.
- Flexible for continuous and discrete problems.

3.15.5 Limitations

- Can require many objective evaluations.
- Requires tuning of population size and mutation rate.
- Can converge slowly.
- Does not guarantee a global optimum.

3.16 Particle Swarm Optimization

Particle swarm optimization also maintains a population of candidate solutions called **particles**.

Each particle uses information about:

- its current location;
- its previous velocity;
- its own best location;
- the best location found by the group.

A simplified velocity update is

$$v_{i,k+1} = wv_{i,k} + c_1r_1(p_i - x_{i,k}) + c_2r_2(g - x_{i,k}),$$

followed by

$$x_{i,k+1} = x_{i,k} + v_{i,k+1}.$$

The intuition is

Momentum + Individual Experience + Group Experience + Randomness.

3.16.1 Advantages

- Does not require derivatives.
- Explores multiple regions simultaneously.
- Relatively intuitive and flexible.

3.16.2 Limitations

- May require many function evaluations.
- Sensitive to tuning parameters.
- Can converge prematurely.
- Global optimality is not guaranteed.

3.17 Deterministic versus Metaheuristic Methods

The two families search very differently.

| Method | Type | Derivatives | Search | Main Advantage | Main Limitation |
|---------------------|---------------|------------------|--------------------|-------------------------------|-----------------------------------|
| Bisection | Deterministic | No | Root finding | Very reliable | Slow |
| Newton | Deterministic | Yes | Local | Very fast near solution | Sensitive to derivative and start |
| Secant | Deterministic | No | Local | Fast without derivative | Less reliable |
| Gradient Descent | Deterministic | Gradient | Local | Simple and scalable | Step size matters |
| BFGS | Deterministic | Usually gradient | Local | Efficient for smooth problems | Can reach local optimum |
| Simulated Annealing | Metaheuristic | No | Global exploration | Can escape local optima | Slow and stochastic |
| Genetic Algorithm | Metaheuristic | No | Population | Flexible search | Computationally expensive |
| Particle Swarm | Metaheuristic | No | Population | Cooperative exploration | Requires tuning |

The important question is not

Which algorithm is the best?

Instead, ask

Which algorithm is appropriate for the structure of the statistical problem?

For a smooth and well-behaved objective,

deterministic methods

are usually preferable.

For a highly multimodal, irregular, or derivative-free objective,

may become useful.

Suppose the objective is smooth, convex, and easy to differentiate.

Would you choose BFGS or simulated annealing?

What changes if the objective has many local minima and no useful derivatives?

3.18 Optimization Transfer

Sometimes the original objective is difficult to optimize directly.

Instead, we construct an easier **surrogate function** and repeatedly optimize the surrogate.

The general idea is

Difficult Objective $\rightarrow$ Easier Surrogate $\rightarrow$ Update $\rightarrow$ Repeat.

This principle is often called **optimization transfer**.

The surrogate should be easier to optimize while still guiding the algorithm toward improvement of the original objective.

We will encounter this idea again in later statistical algorithms.

3.19 Computational Perspective

The goal is not simply to memorize optimization algorithms.

For a statistical problem, ask:

1. What is the statistical quantity we want?
2. Can it be written as an optimization or root-finding problem?
3. Is there a closed-form solution?
4. Is the parameter scalar or multivariate?
5. Is the objective smooth?
6. Are derivatives available?
7. Are there many local optima?
8. How sensitive is the method to the starting value?
9. Is the computation numerically stable?
10. How can we validate the result?

This brings us back to the central idea of statistical computing:

$$\boxed{\text{Theory} \iff \text{Computation} \iff \text{Practice}.}$$

3.20 Questions to Think at Home

1. Why can a mathematically correct optimization formula still lead to a poor computational implementation?
2. Why is a closed-form solution useful even when numerical optimization will eventually be used?
3. Why is bisection more reliable than Newton's method, and what is the computational cost of that reliability?
4. What additional information does Newton's method use compared with bisection?
5. Why can different starting values lead to different solutions for the same objective function?
6. What role does the step size play in gradient descent?
7. Why might poor scaling make optimization difficult?
8. Why is solving

$$Hd = -\nabla Q$$

usually preferable to explicitly computing H^{-1} ?

9. Why can BFGS be useful when the Hessian is expensive to compute?
10. Why might a metaheuristic algorithm deliberately accept a worse solution?
11. When would you prefer simulated annealing over a deterministic local optimizer?
12. What is the main difference between simulated annealing, genetic algorithms, and particle swarm optimization?
13. Why does fewer iterations not necessarily imply lower computational cost?
14. How are optimization, likelihood estimation, and statistical inference connected?

3.21 Takeaways

- Many statistical estimators are solutions to optimization problems.
- Closed-form solutions are convenient but often unavailable.
- Numerical optimization connects statistical theory with practical computation.
- Root finding is closely related to optimization.
- Bisection is reliable but relatively slow.
- Newton's method is fast near a solution but uses derivative information.
- The secant method approximates derivative information numerically.
- Multivariate optimization replaces derivatives with gradients and Hessians.
- Gradient descent uses local first-order information.
- Newton's method uses curvature information through the Hessian.
- BFGS approximates curvature without repeatedly computing the full Hessian.
- Scaling can strongly affect computational performance.
- Deterministic methods are usually efficient for smooth, well-behaved objectives.
- Metaheuristic methods trade speed and predictability for broader exploration.
- Simulated annealing can escape local optima by occasionally accepting worse solutions.
- Genetic algorithms and particle swarm optimization use populations of candidate solutions.
- There is no universally best optimization algorithm.
- A good computational method balances accuracy, stability, speed, and exploration.

Examples are adapted from the following sources:

- Peng, R.D. [Advanced Statistical Computing](#).

Part I

Assignment

Assignment 1

Due date: Friday, September 11, 2026

Total: 30 points

Submission

For this assignment, please use **R Markdown** or **Quarto**.

Submit both:

1. your source file (`.Rmd` or `.qmd`), and
2. the corresponding rendered PDF.

For multiple-choice questions, clearly indicate your selected answer.

For computational questions, include the R code used to obtain your answer.

Question 1: R Programming and Statistical Computing [10]

Select the **best answer**. Each question is worth **1 point**.

1a)

Which of the following creates a numeric vector in R?

- A. `x <- list(1, 2, 3)`
- B. `x <- c(1, 2, 3)`
- C. `x <- data.frame(1, 2, 3)`
- D. `x <- matrix(1, 2, 3)`

1b)

Which expression correctly returns the mean of `x` after removing missing values?

- A. `mean(x, remove = TRUE)`
- B. `mean(x, missing = FALSE)`
- C. `mean(x, na.rm = TRUE)`
- D. `mean(na.omit = x)`

1c)

Suppose

$$x = (10, 20, 30, 40).$$

What does the following R expression return?

```
x[c(1, 3)]
```

- A. 10 20
- B. 10 30
- C. 20 40
- D. 30 40

1d)

Which statement about R indexing is correct?

- A. R indexing starts from 0.
- B. R indexing starts from 1.
- C. R indexing depends on the data type.
- D. R indexing starts from -1 .

1e)

Which function is most appropriate for applying a function to the **columns of a numeric matrix**?

- A. `apply()`
- B. `lm()`
- C. `filter()`
- D. `rbind()`

1f)

What is the main advantage of using `vapply()` instead of `sapply()`?

- A. It always runs in parallel.
- B. It requires a prespecified output type.
- C. It works only on matrices.
- D. It automatically removes missing values.

1g)

Which of the following is the native R pipe operator?

- A. `%>%`
- B. `->`
- C. `|>`
- D. `%*%`

1h)

Which `dplyr` function is used to retain rows satisfying a condition?

- A. `select()`
- B. `filter()`
- C. `mutate()`
- D. `summarise()`

1i)

Which statement concerning `apply()` and a `for` loop is most accurate?

- A. `apply()` is always faster than a `for` loop.
- B. A `for` loop is always faster than `apply()`.
- C. Neither is always faster; performance depends on the computation.
- D. `apply()` performs computations in parallel by default.

1j)

Why is reproducibility important in statistical computing?

- A. It guarantees that every statistical model is correct.
 - B. It allows an analysis to be rerun and verified.
 - C. It eliminates sampling variability.
 - D. It eliminates numerical error.
-

Question 2: Computational Results Using the Iris Data [10]

Use the built-in `iris` dataset in R.

For each computational question, include the R code used to obtain your answer.

2a) [2]

Report the sample mean of `Sepal.Length`, rounded to three decimal places.

2b) [2]

Report the sample standard deviation of `Sepal.Width`, rounded to three decimal places.

2c) [2]

How many observations in the `iris` dataset belong to the species `virginica`?

- A. 25
- B. 40
- C. 50
- D. 75

2d) [2]

Fit the linear regression model

$$\text{Sepal.Length} = \beta_0 + \beta_1 \text{Sepal.Width} + \varepsilon.$$

Report the estimated slope coefficient $\hat{\beta}_1$, rounded to three decimal places.

2e) [2]

Based on the fitted model in part (d), which statement is correct?

- A. Sepal length tends to increase as sepal width increases.
- B. Sepal length tends to decrease as sepal width increases.
- C. The estimated slope is exactly zero.
- D. The model cannot be fitted because both variables are continuous.

Question 3: R Coding and Visualization [6]

Use the `iris` dataset for both parts.

3a) [3]

Using `ggplot2`, construct a scatter plot with:

- `Sepal.Width` on the x -axis,
- `Sepal.Length` on the y -axis,
- points coloured according to `Species`.

Your figure must contain:

1. appropriate axis labels,
2. an informative title, and
3. a readable legend.

Include both your R code and the resulting figure.

3b) [3]

Add a linear regression line to the plot from part (a).

Use

```
geom_smooth(method = "lm", se = FALSE)
```

and include the final plot in your report.

Question 4: Numerical Linear Algebra [4]

4a) [2]

Suppose Σ is a positive-definite covariance matrix with Cholesky decomposition

$$\Sigma = R^\top R,$$

where R is upper triangular.

Which expression correctly computes $\log \det(\Sigma)$?

A.

$$\sum_i \log(R_{ii})$$

B.

$$2 \sum_i \log(R_{ii})$$

C.

$$\sum_i R_{ii}^2$$

D.

$$2 \log \left(\sum_i R_{ii} \right)$$

4b) [2]

Let

$$R = \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}, \quad \Sigma = R^T R.$$

Compute $\log \det(\Sigma)$.

- A. $\log(6)$
- B. $\log(12)$
- C. $2 \log(6)$
- D. 6
